



计算机组成与设计

Computer Organization & Design The Hardware/Software Interface

Preface

Ma De 马德

**College of Computer Science and Technology
Zhejiang University
made@zju.edu.cn**



Curriculum System: 三位一体、循序递进

立足基础、加强实践、服务专业、进入国际

- 《**数字逻辑课程**》：计算机组成相关部件的设计
 - 组合电路设计、时序电路设计
- 《**计算机组成**》：设计简单**RISC-CPU**核
 - ALU部件
 - 单周期实现、多周期实现简单的32位-CPU
 - RISC
 - 写入FPGA，用实验板卡做测试验证
 - 简易计算机系统（微控制系统）
- 《**计算机系统结构**》：设计流水线**RISC-CPU**核心

基础



核心



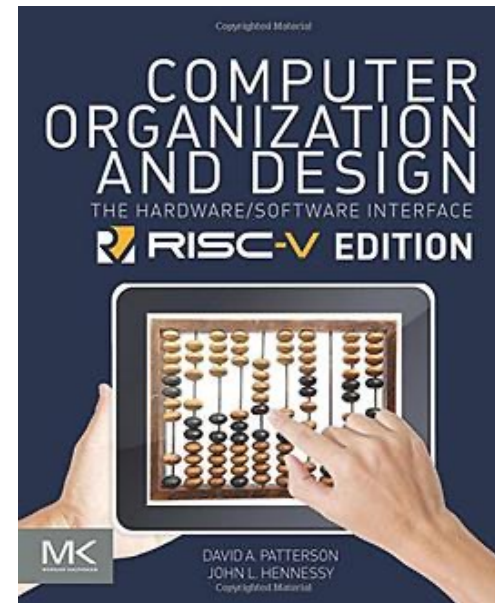
提高

Curriculum Text Book

Computer Organization & Design (MIPS, ARM, RISC-V) ——The Hardware/Software Interface

John L. Hennessy (Stanford University)

David A. Patterson (University of California, Berkeley)





Course Grading

- 平时占 30%
 - 作业、课堂、QUIZ
- 期末闭卷(英文)考试 40%
- 实验占 30%
 - Lab00~03: 基本实验 30%
 - Lab04 :单周期CPU 30%
 - Lab05 :流水线CPU 30%
 - Lab06 :数据缓存 10%
 - 附加分奖励: 优秀作品 (附加后不超过100分)

注: 卷面成绩不到45分(满分100分)者, 总评不及格。



Schedule of experiments

1	Lab00: Vivado介绍和使用	Vavado介绍和使用, MUX模块封装
2	Lab01: Warmup	ALU 和Regfile设计
3	Lab02: CPU实验环境搭建	IP核集成SOC设计, 完成CPU测试环境搭建
4	Lab03: 复杂操作实现	乘除法、浮点加法实现
5	Lab04: 单周期CPU设计	1. CPU核集成 2. CPU数据通路 3. CPU控制器 4. 指令扩展 5. 中断
6	Lab05: 流水线CPU设计	1. 流水线处理器集成 2. IF-ID设计与集成 3. EX-MEM-WB设计与集成 4. 冒险与stall
7	Lab06: Data Cache	1. 数据高速缓存的设计



计算机组成与设计

Computer Organization & Design The Hardware/Software Interface

Computer abstractions and Technology

Ma De 马德

College of Computer Science and Technology
Zhejiang University
made@zju.edu.cn



Outline

- ❑ Introduction
- ❑ Computer organization
- ❑ How build processors ?
- ❑ Computer design: performance and idea
- ❑ What you can learn from this course ?

Outline



- Introduction
- Computer organization
- How build processors ?
- Computer design: performance and idea
- What you can learn from this course ?



Computer Generations

- ❑ **Pre-computer – ~ 1946**
 - Non-electrical, non-programmable (非电子, 不可编程)
- ❑ **First Generation – 1946 ~ 1956**
 - Vacuum tubes, programmable (电子管, 可编程, 图灵完全)
- ❑ **Second Generation – 1956 ~ 1964**
 - Transistor, programming languages (晶体管, 编程语言开始应用)
- ❑ **Third Generation – 1964 ~ 1971**
 - Integrated Circuit, OS (集成电路, 操作系统开始应用)
- ❑ **Fourth Generation – 1971 ~ now**
 - Microprocessor, GUI, Personal Computer
(大规模集成微处理器, 图形用户界面, 个人电脑兴起)
- ❑ **Fifth Generation – future**
 - Still in development

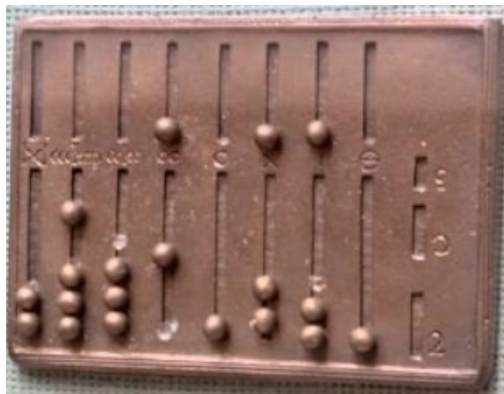
Pre-computers: 计数的辅助工作



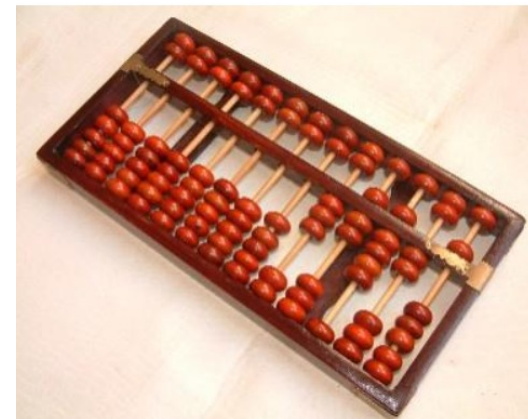
Digit: Latin for fingers



Tally sticks



Roman abacus



Asian abacus

The First Mechanical Calculating Machine

The 17th century: the beginning of mechanical calculators

17世纪：机械式计数器开始出现

Pascaline (Pascal's calculator)

- Invented by Blaise Pascal in 1642-1645 (**19-22岁！**)
- Addition and Subtraction: Two basic operations
- Multiplication and division: through repeated addition or subtraction

两个基本操作：加与减

乘与除这两个操作通过重复地加或减间接实现



Blaise Pascal (1623-1662)

Pascal编程语言取名就是为了纪念Blaise Pascal



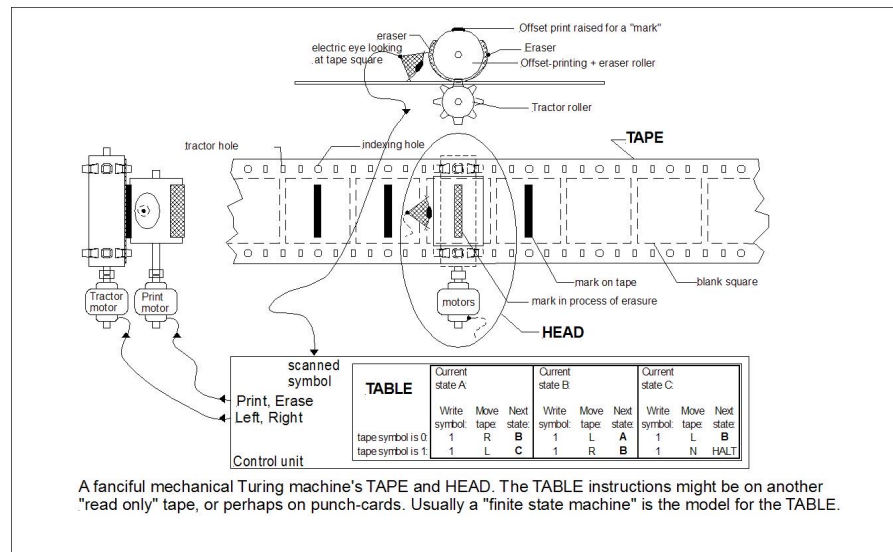
Pascaline (1645)

Turing Machine (1936)

Alan Turing described a theoretical device called the **Turing machine**, formalized the concepts of **computation and algorithms in 1936**



A.M.Turing (1912-1954)



Turing machine as a mechanical device

First electronic digital computing device



Atanasoff-Berry Computer(ABC) 1942

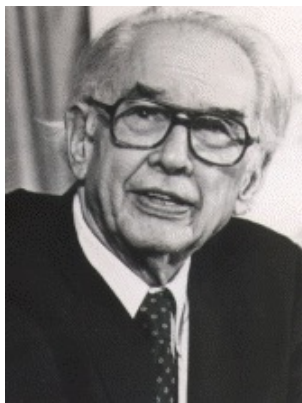
- First electronic digit computing device 第一台纯电子的数字计算设备
- Invented by Professor **John Atanasoff** and graduate student Clifford Berry at Iowa State College between **1939 and 1942**
- Special purpose. **neither programmable, nor Turing-complete**
非常接近现代计算机，但**不可编程、非图灵完全**



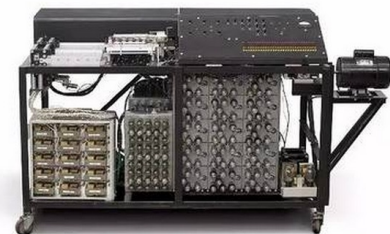
Add-subtract module
加减运算模块

对现代计算机发展的重要贡献

- 使用二进制数表示
- 用纯电子实现算术与逻辑运算
- 计算和存储分离

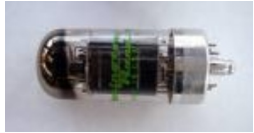


John Atanasoff (1903-1995) Clifford Berry (1918-1963)

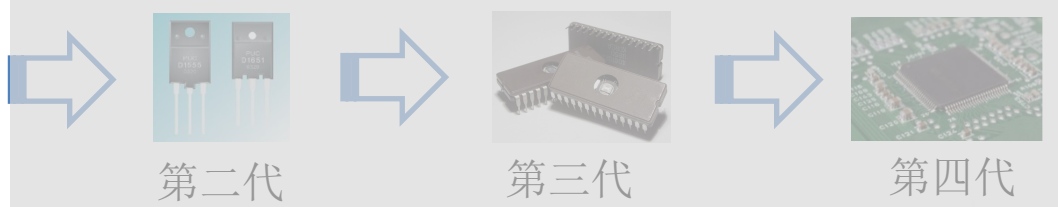


Atanasoff-Berry Computer
(1942)

The First Generation (1946-1956): Vacuum Tubes



第一代



第二代

第三代

第四代

□ 基本特征：运算用电子管(即真空管)实现、可编程

vacuum tubes for circuitry and magnetic drums for memory, programmable

□ Examples:

- ENIACS
- EDSAC
- UNIVAC I, UNIVAC II, UNIVAC 1101



Vacuum tubes

First general-purpose electronic digital computer



ENIAC (Electronic Numerical Integrator and Computer) in 1946

- ❑ First general-purpose electronic computer **第一台通用电子计算机**
- ❑ Designed by **John Mauchly** and **J. Presper Eckert** of the University of Pennsylvania
- ❑ General-purpose instruction set, decimal, programmable & Turing-complete

通用指令集、十进制、可编程、图灵完全

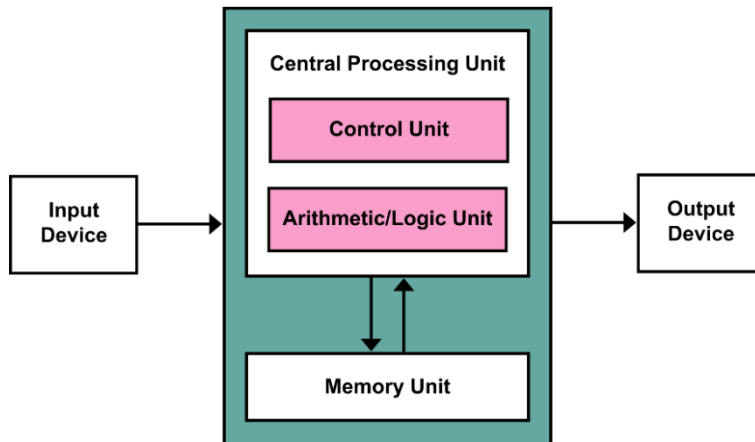


von Neumann architecture (1945)

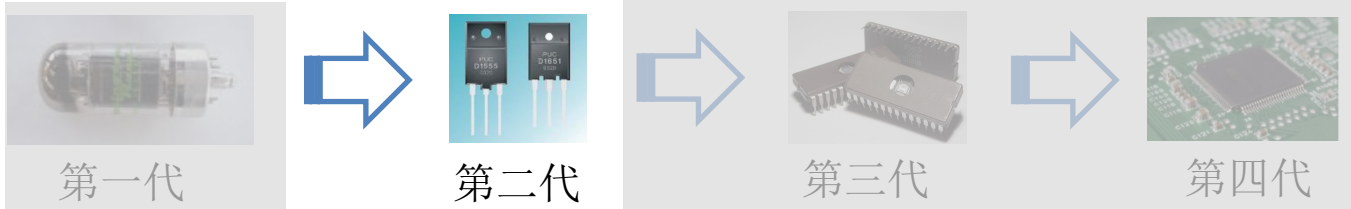
- Computation and memory are separated
计算与存储分离
- Memory that stores data and instructions
数据与指令保存在同一个存储器
- Input and output mechanisms
- Instruction set architecture



John von Neumann (1903-1957)



The Second Generation (1956-1964): Transistors



□ 基本特征：晶体管替代电子管

Transistor replace Vacuum tubes

- Smaller, faster, less power, more reliable

体积减小、速度加快、计算更可靠

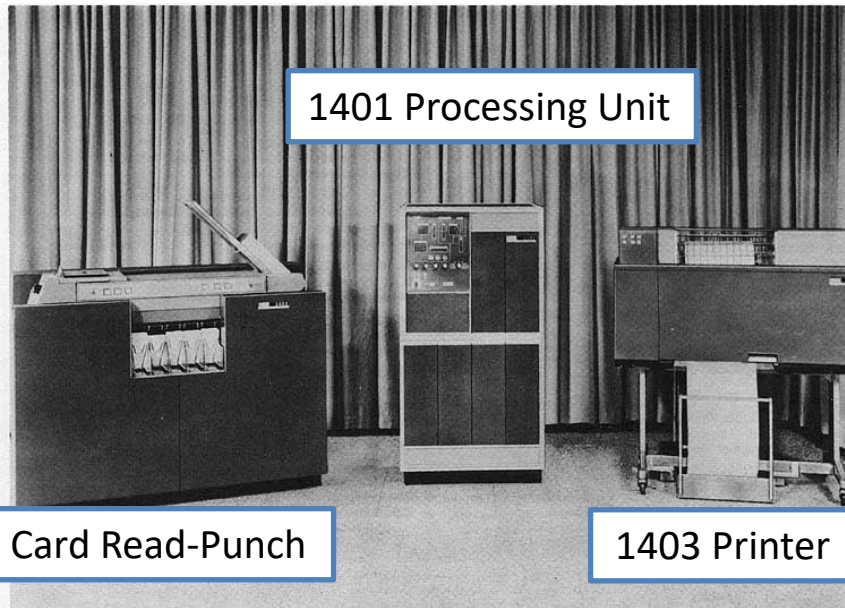
- Programming: from binary machine language to symbolic languages
- I/O: Punched cards for input and printouts for output.
- Examples: UNIVAC III, RCA 501, Philco TransactS-2000, NCR 300 series, IBM7030 Stretch



Transistors
晶体管

IBM 1401 (1959)

- announced by IBM on October 5, 1959.
- a variable-wordlength decimal computer **(可变字长、十进制)**
- installed more than 10,000 units between 1960 and 1964. **商业上大获成功**



IBM 1401 system



IBM 1401

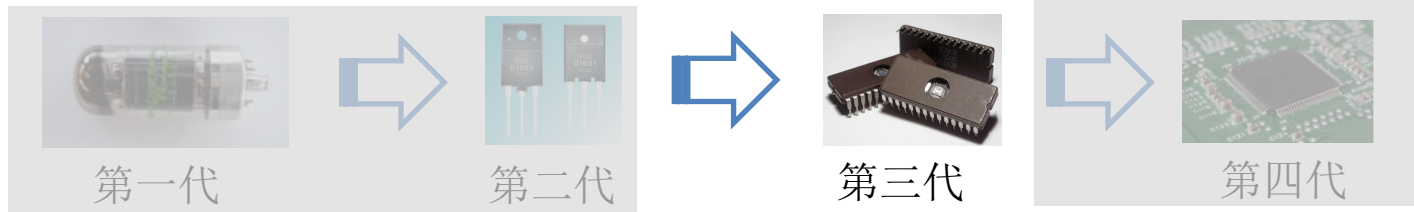
UNIVAC III (1962)

■ UNIVersal Automatic Computer III (1962)

- Designed by **John Perper Eckert** and **John Mauchly** (ENIAC designers)
- An improved transistorized replacement for UNIVAC I and UNIVAC II



The Third Generation (1964-1971): Integrated Circuits



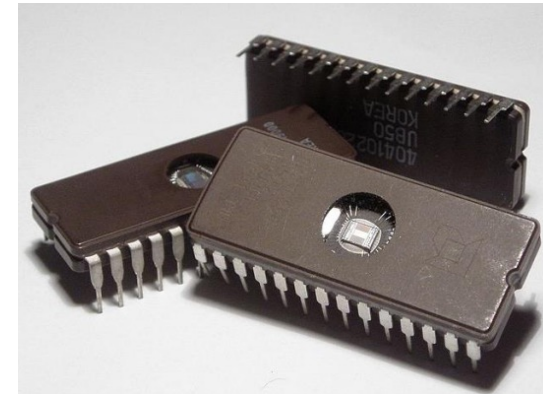
□ 基本特征：集成电路替代晶体管

Integrated Circuits Replaced Transistors

- smaller and cheaper than their predecessors (**更小、更便宜**)
- Electronic components on single chips of silicon

□ Operating systems: run many different applications at one time

□ Examples: IBM System/360, Honeywell 200



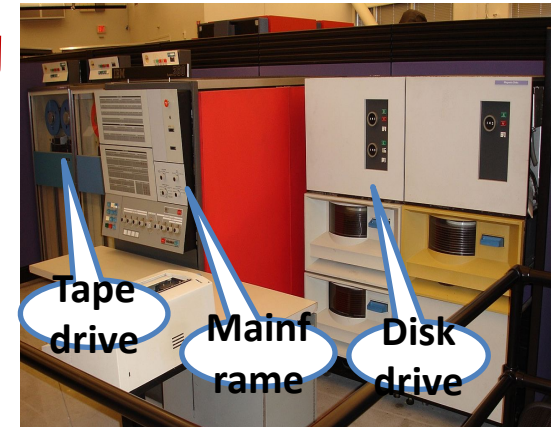
IBM System/360 Family (1964)

■ Announced in 1964, delivered in 1965

- Model 30: up to 34,500 instructions per second, with memory from 8 to 64 KB.
- Model 91 in 1967: up to 16.6 million instructions per second, with memory from 1M to 4M
- Extremely successful in the market **商业上极为成功**

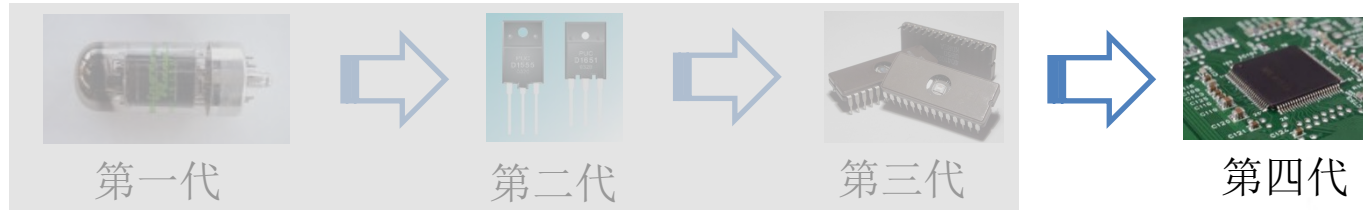
■ Backward compatibility **向前兼容**

- **Computer System Architecture Specification**
 - Chief Architect: Gene Amdahl (NAE in 1967)
- **A Single Operating System for All: OS/360**
 - Dynamic address translation (virtual memory) **虚拟内存**
 - Time-sharing **分时**



IBM System/360 Model 30

The Fourth Generation (1971-now): Microprocessors



□ 基本特征：微处理器替代集成电路

Microprocessors replaced Integrated Circuits

- smaller, cheaper, more powerful

□ Microprocessor: entire CPU fits on a single chip.



- Three companies developed the microprocessor independently at the same time: Texas Instruments, Intel, and Garrett AiResearch **in 1971**

□ Emerging of Personal Computers 个人计算机兴起

Dynabook vision (1972)

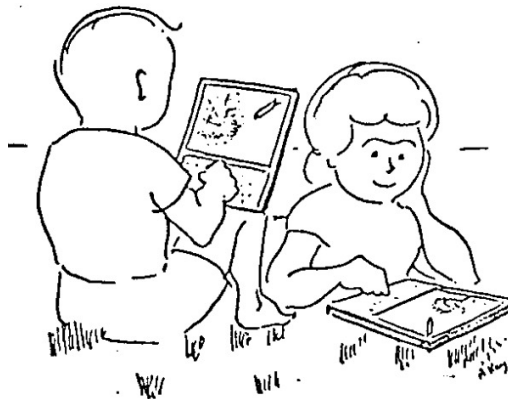
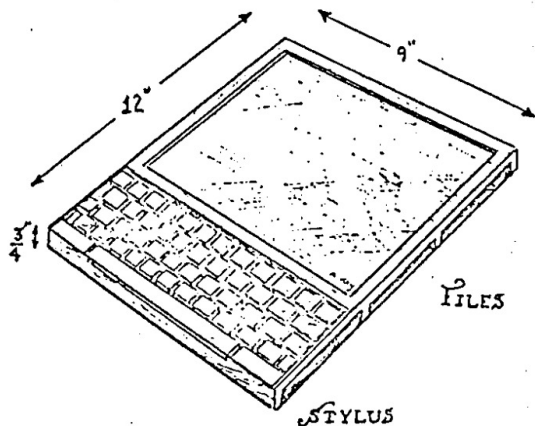
■ Dynabook

- In 1972 [Alan Kay](#)'s article "A personal computer for children of all ages"
- Conceptual vision of a laptop/tablet computer.

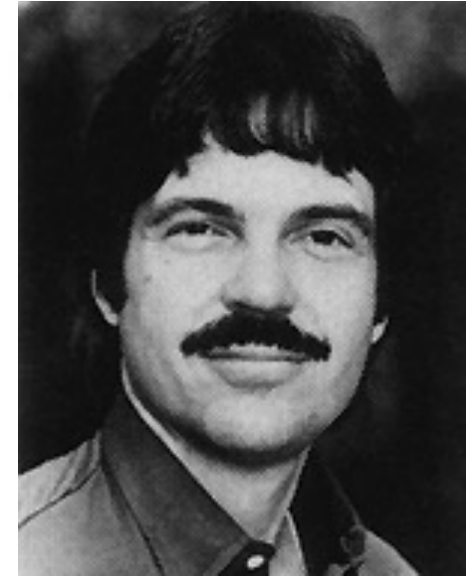
提出了便携电脑与平板电脑的原型设计

A Personal Computer for Children of All Ages

Alan C. Kay
Xerox Palo Alto Research Center



Turing Award 2003



Alan Kay (1940-)

Also:

- Smalltalk面向对象程序设计思想提出者
- overlapping windowing GUI提出者

The first modern personal computer (首台现代个人电脑)



Xerox Alto in 1973 at Xerox PARC

- designed mostly by Charles P. Thacker
- Mouse-driven Graphical User Interface

Turing Award 2009

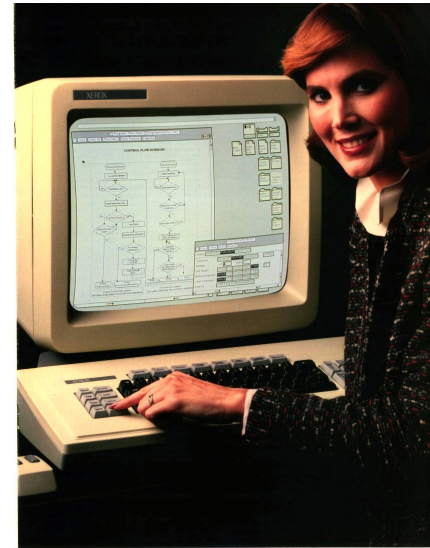


Charles Thacker (1943-2017)



Xerox Alto (1973)

Alto Mouse



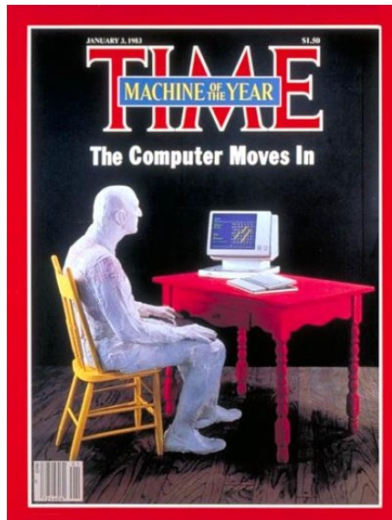
Xerox Star (1981)

Commercial Personal Computer

■ IBM PC: with an Intel CPU & Microsoft's DOS in 1981

- Price started at \$1,565
- 300,000 sold in 1981
- **3,274,000 sold in 1982**

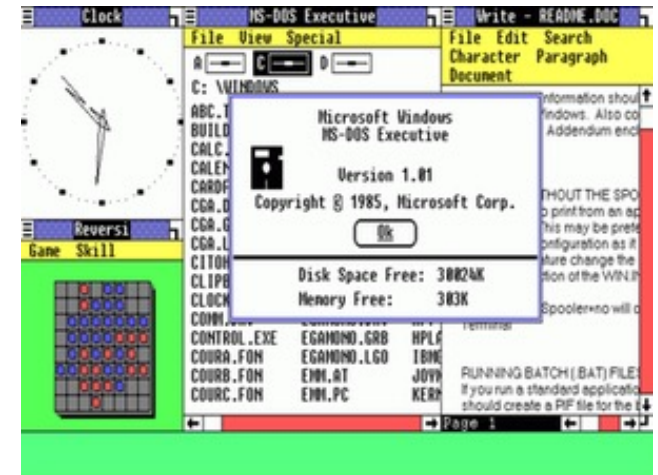
■ Apple (Steve Jobs) developed Macintosh OS with GUI in 1984



TIME named in 1982

Machine of the year

■ Microsoft (Bill Gates) released Windows 1.0 in 1985, the first version.



RISC Architecture (1980s) : 更高的性能



■ RISC (Reduced Instruction Set Computer): a computer instruction set

- fewer cycles per instruction (CPI) than a Complex Instruction Set Computer (CISC).

指令执行用尽量少的时钟周期、指令编码长度定长等 → 提高CPU与编译效率

■ Now used in 99% of new computer chips, e.g. smartphones

现在绝大部分芯片已采用RISC架构，如ARM芯片、智能手机

Turing Award 1987

1975-1980: IBM 801
The 1st RISC system

→ IBM [PowerPC](#)

■ spent his entire career as an industrial researcher for IBM in 1956-1992.



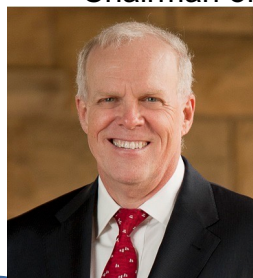
John Cocke
(1925-2002)

Turing Award 2017

MIPS project of a
[Stanford graduate course](#)
in 1981

→ found [MIPS](#) company
in 1985

- President of Stanford Univ. (2000-2016)
- Chairman of Alphabet



John L. Hennessy
(1952-)

Turing Award 2017

[Berkeley RISC](#) project
from 1980

→ commercialized as
the [SPARC](#).

- one of the innovators of RAID in 1989-1993



David Patterson
(1947-)



The Computer Revolution

- **Progress in computer technology**
 - **Underpinned by Moore's Law**
- **Makes novel applications feasible**
 - **Computers in automobiles**
 - **Cell phones**
 - **Human genome project**
 - **World Wide Web**
 - **Search Engines**
- **Computers are pervasive**



Classes of Computers

■ Personal computers

- General purpose, variety of software
- Subject to cost/performance tradeoff

■ Server computers

- Network based
- High capacity, performance, reliability
- Range from small servers to building sized

■ Supercomputers

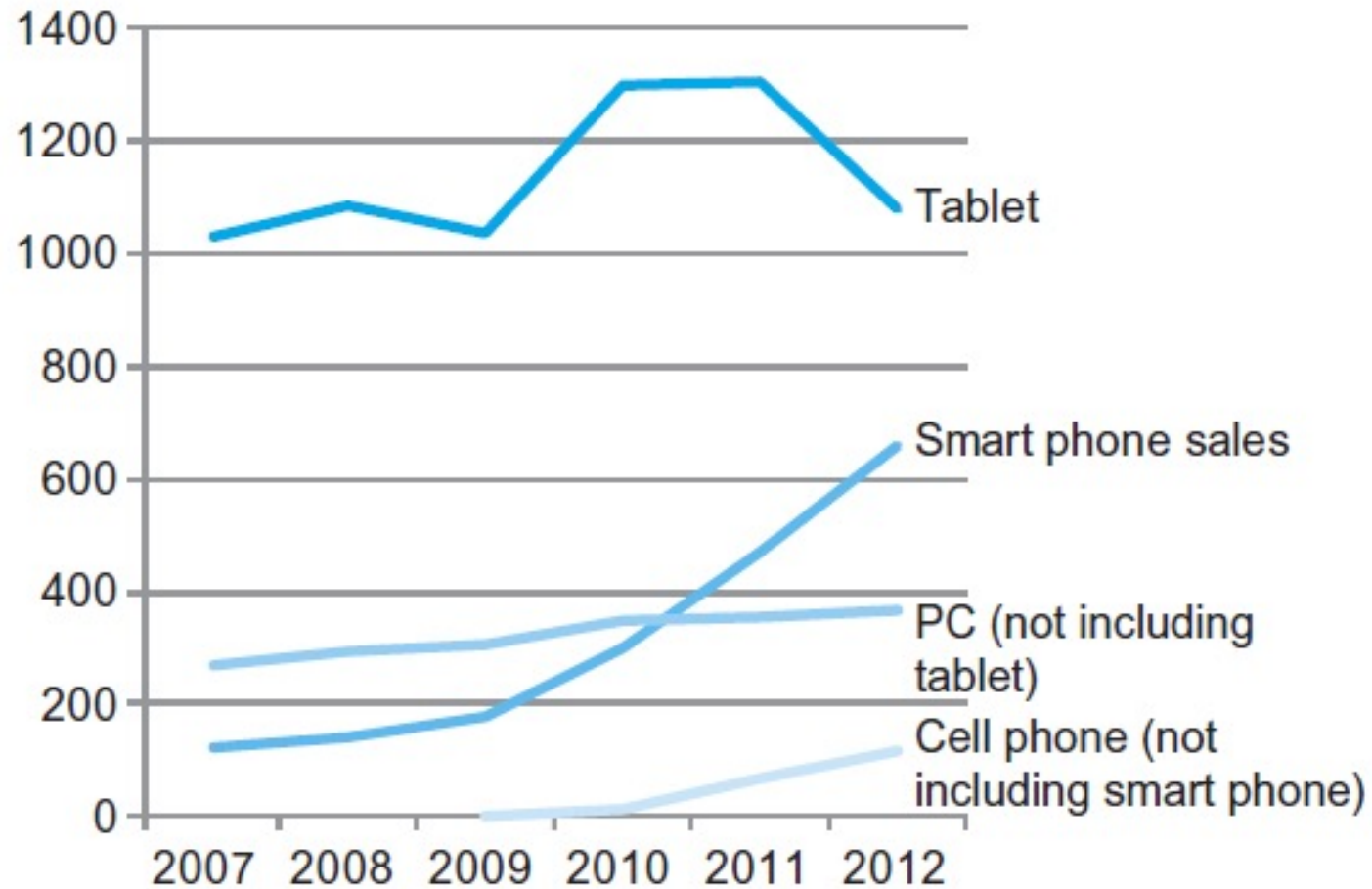
- High-end scientific and engineering calculations
- Highest capability but represent a small fraction of the overall computer market

■ Embedded computers

- Hidden as components of systems
- Stringent power/performance/cost constraints



The PostPC Era





The PostPC Era

■ Personal Mobile Device (PMD)

- Battery operated
- Connects to the Internet
- Hundreds of dollars
- Smart phones, tablets, electronic glasses

■ Cloud computing

- Warehouse Scale Computers (WSC)
- Software as a Service (SaaS)
- Portion of software run on a PMD and a portion run in the Cloud
- Amazon ,Google , Alibaba, Huawei

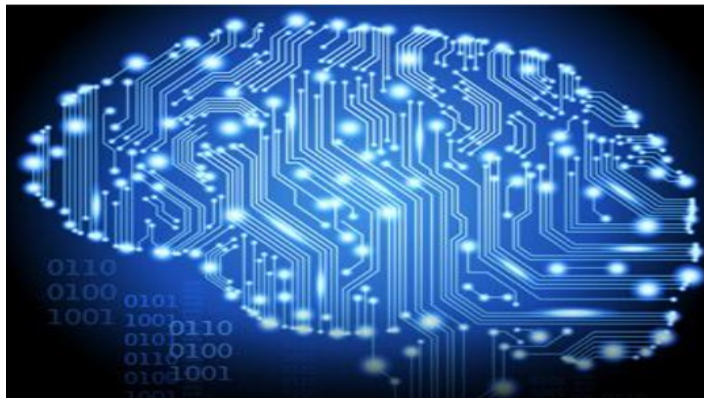
The Fifth Generation?



Quantum Computer 量子计算机



Wearable Computer 穿戴计算机



Neuromorphic Computer 类脑计算机



Intelligence Computer 人智计算机



What is a computer?

- ❑ Computer is an electronic device that manipulates data according to a list of **instructions** (program), with capability of Turing machine.
 - Electronic realization **电子化的实现方式**
 - A **set of instructions** in a well-defined manner **有指令集** → general-purpose
 - Execution of a **pre-recorded list of instructions** **可执行指令** → program-controlled
 - Memory that can **store** instructions and data **可存储指令与数据** → stored program
 - **Turing-complete** in theory **计算能力上是图灵完全** → equivalent to Turing machine



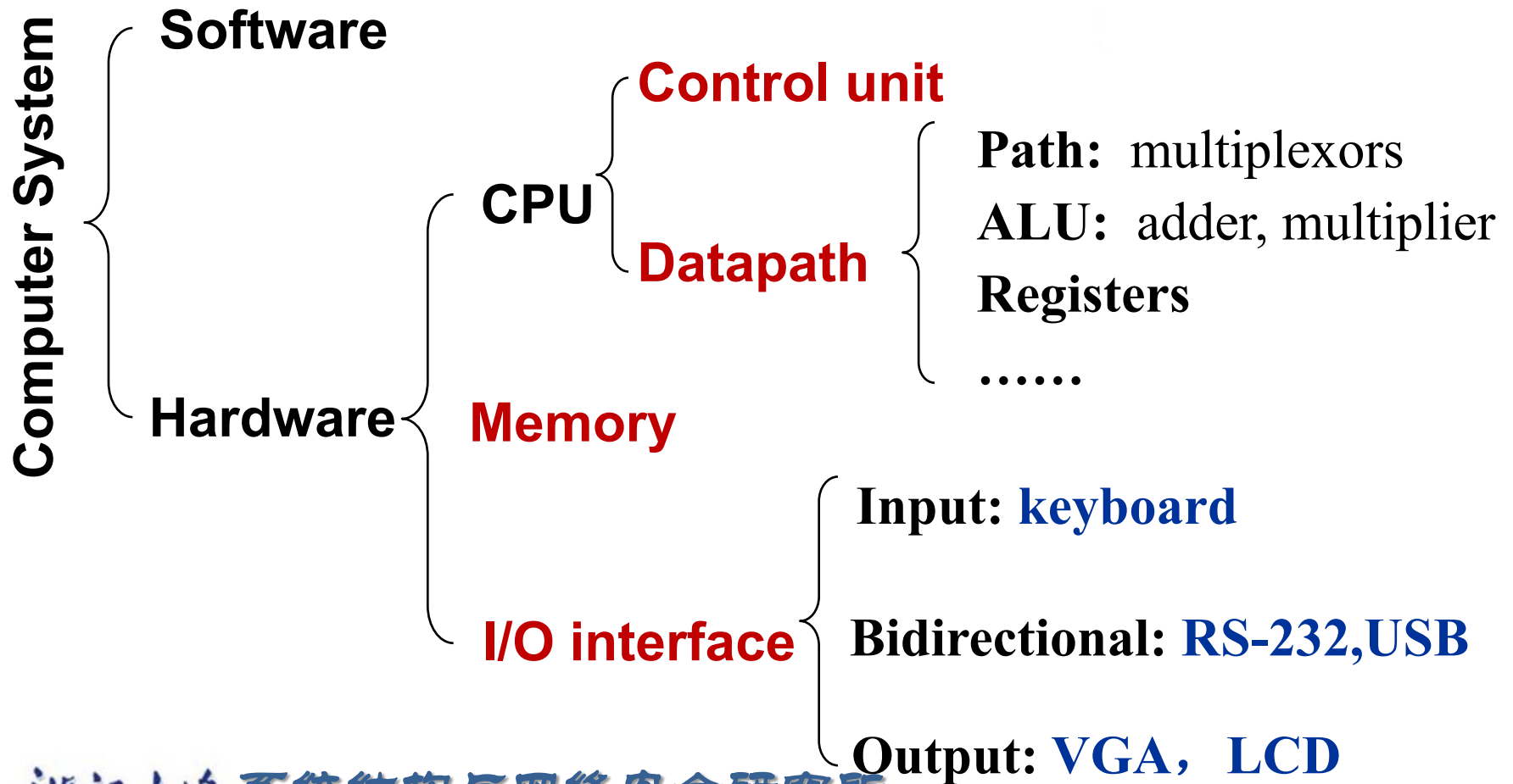
Introduction

- Introduction
- Computer organization
 - hardware
 - software
- How build processors ?
- Computer design: performance and idea
- What you can learn from this course ?

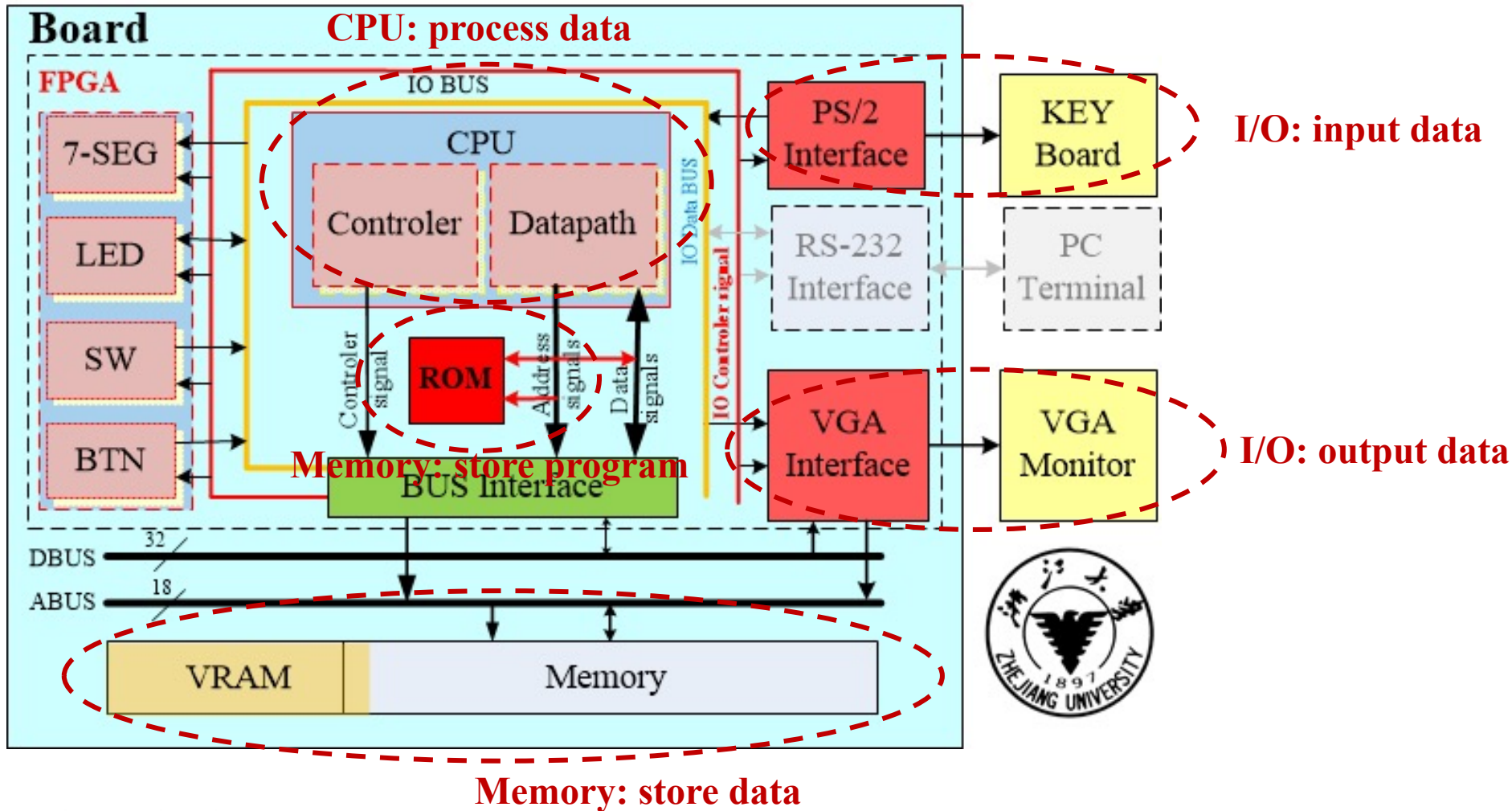


Computer organization

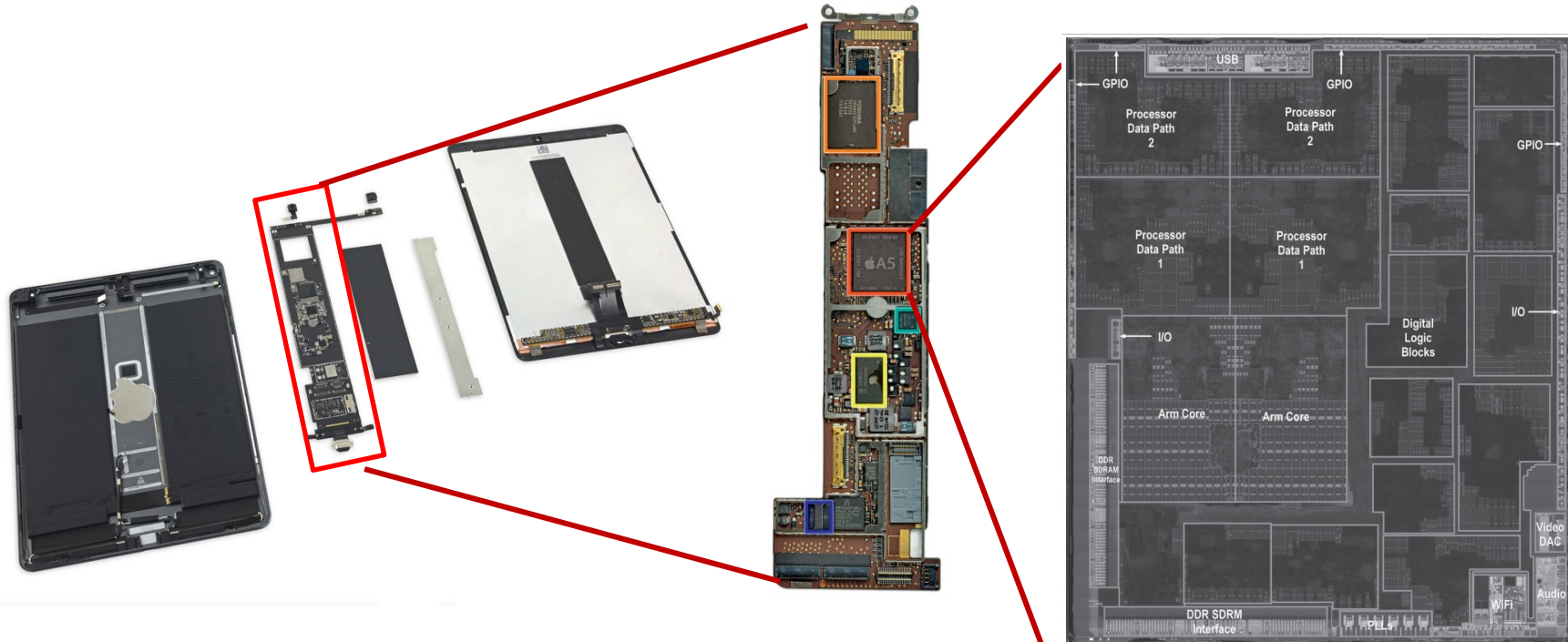
Composability of computer systems



Five classic components of hardware



Opening the Box



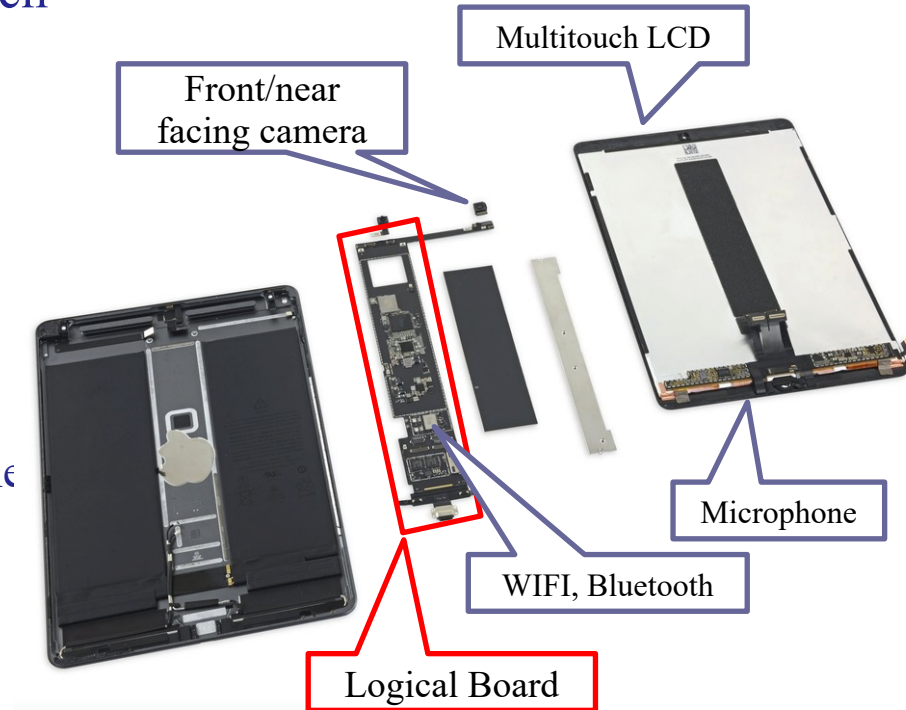
Components of Apple iPad

Logical Board (main board: 主板)
Integrated Circuit on it

Central Processor Unit (CPU)

I/O Interfaces

- ❑ I/O dominates this device.
- ❑ **Input device:** A mechanism through which the computer is fed information
 - Multitouch LCD(触摸屏)
 - Front/near facing camera(前置摄像头)
 - Microphone, headphone(麦克风)
 - Gyroscope(陀螺仪)
 - WIFI, Bluetooth(蓝牙)
- ❑ **Output device:** A mechanism that conveys the result of a computer to a user, such as a displayer, or to another computer
 - Speaker(扬声器)
 - Multitouch LCD
 - WIFI, Bluetooth

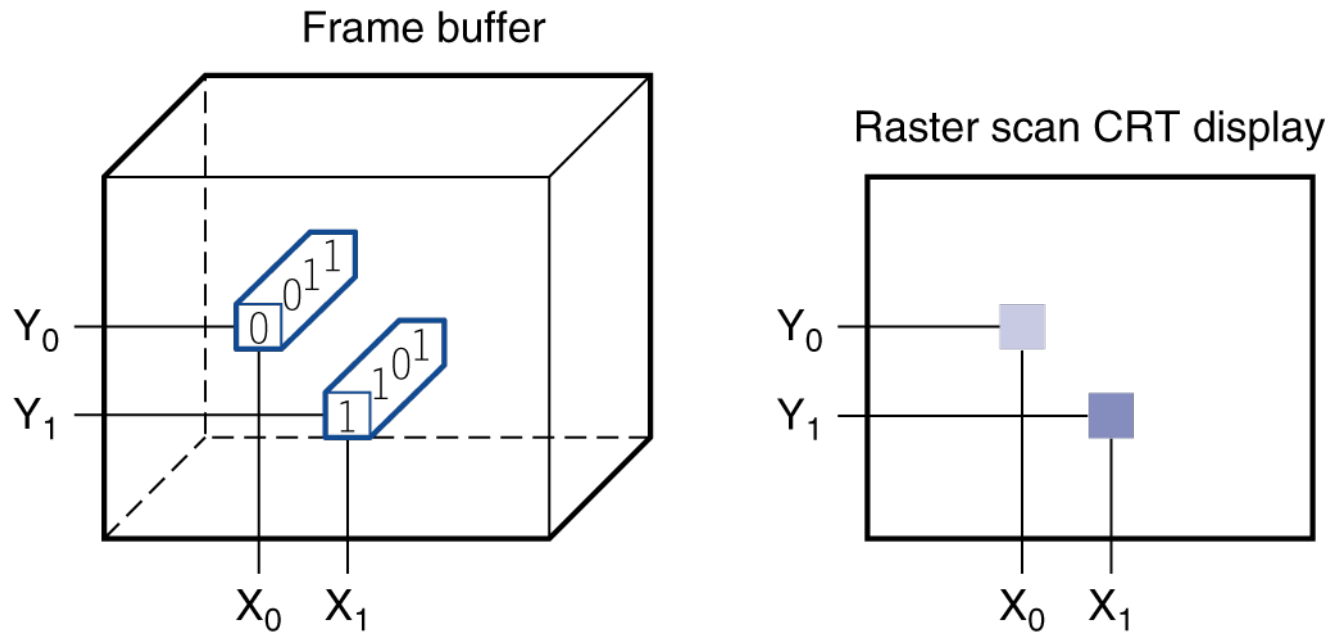


Components of Apple iPad

Through the Looking Glass

□ LCD screen: picture elements (pixels)

- Mirrors content of frame buffer memory

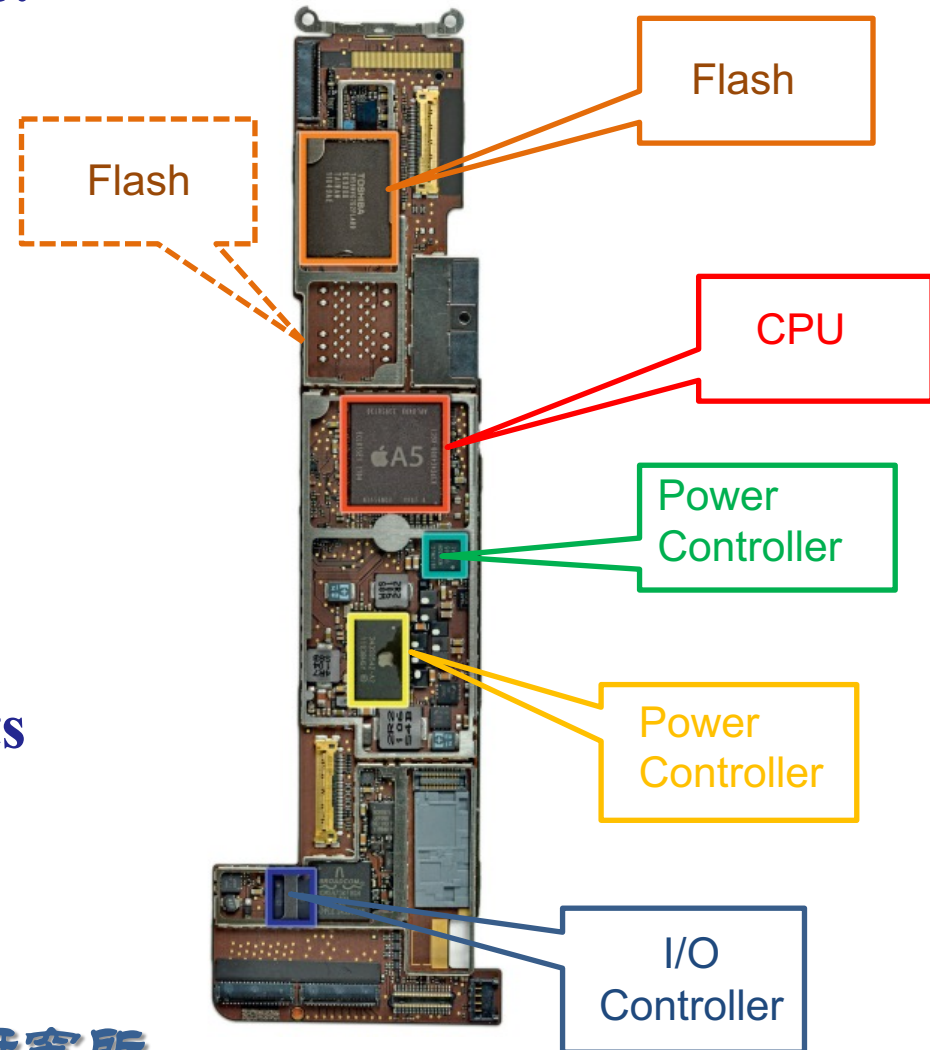


Logical board(主板)

□ Composed of integrated circuits:

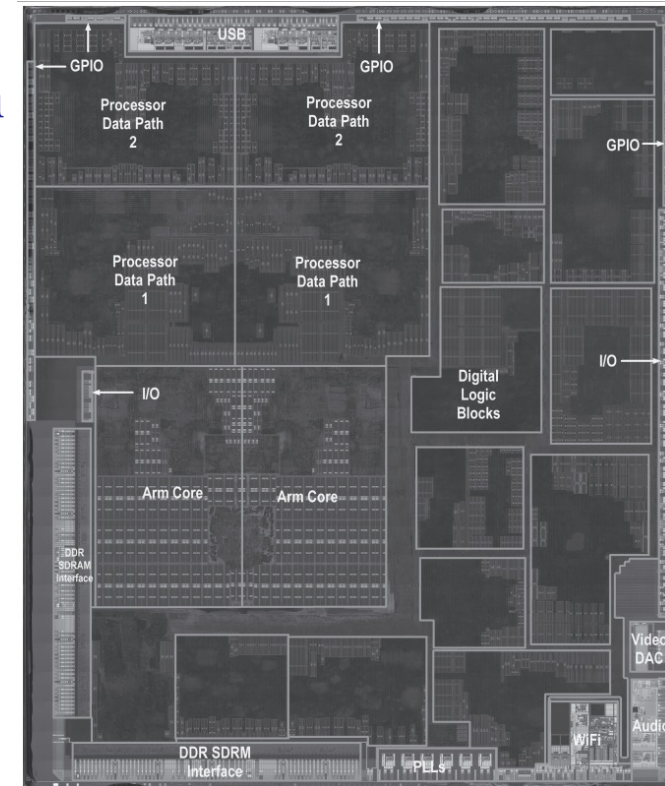
- CPU(Apple A5): Dual ARM processor, 1GHZ, 512MB DDR inside package.
- Flash(闪存): 32GB Flash.
- Power controller: start up(启动), shutdown(关机), standby(待机)
- I/O controller: e.g. Control Touchscreen , control microphone

□ Composed of integrated circuits (chips: 集成电路, 芯片): A device combining dozens of millions of transistors.



CPU (Processor)

- **CPU(Processor):** active part of the computer, which contains the **datapath** and **control** and which adds numbers, tests numbers, signals I/O devices to activate, and so on.
 - **Datapath (数据通路):** performs arithmetic operation
 - **Control (控制通路):** commands the datapath, memory, and I/O devices according to the instructions of the program





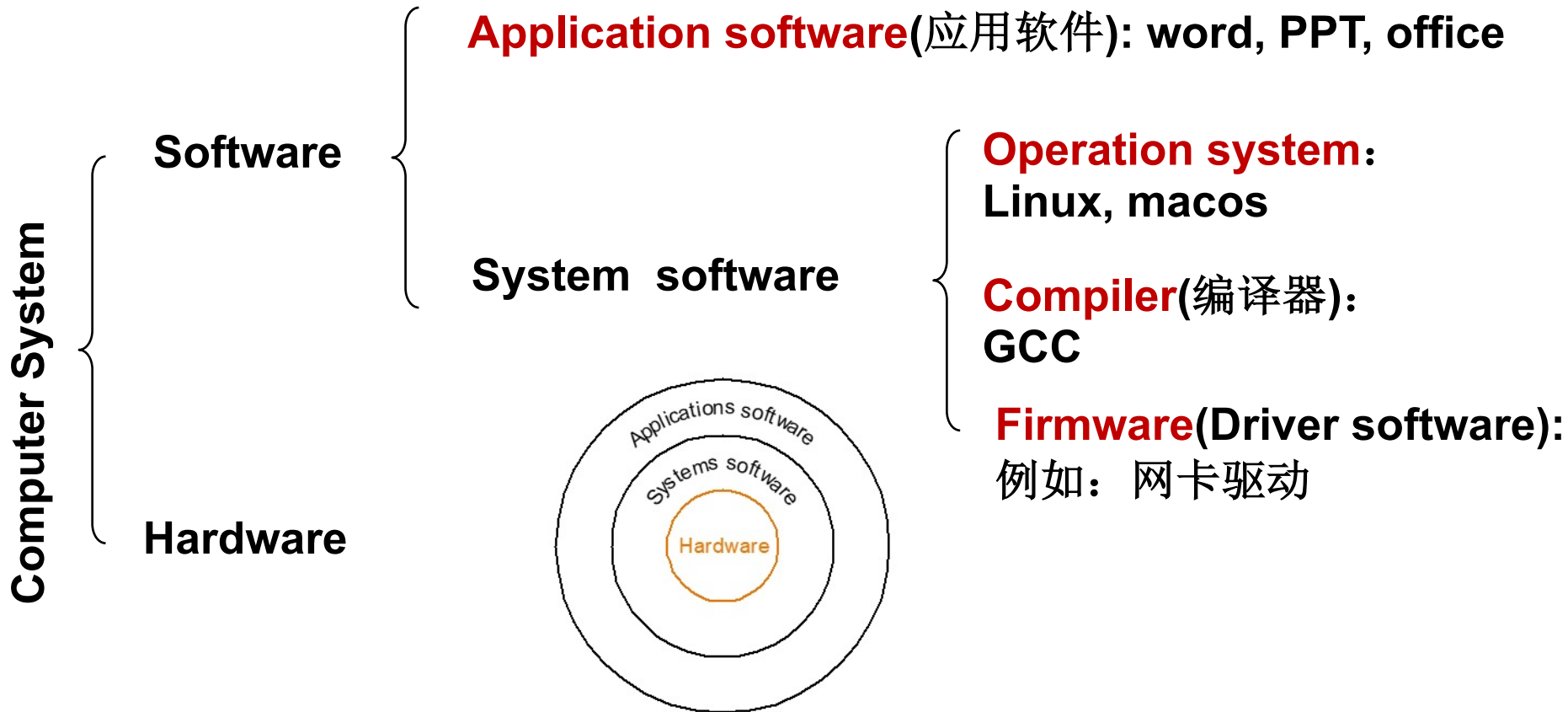
Memory: A safe place for data

- ❑ **Memory(存储):** the storage area programs are kept and that contains the data needed by the running programs
- ❑ **Main Memory(主存): volatile;** used to hold programs while they are running.(e.g. DRAM in computers)
- ❑ **Second memory: nonvolatile;** used to store programs and data between runs. (Flash in PMD, magnetic disks)
- ❑ **Volatile (易失性)**
 - DRAM (Dynamic Random-Access Memory):动态随机存储器
 - SRAM (Static Random Access Memory): 静态随机存储器
- ❑ **Nonvolatile (非易失性)**
 - Solid state memory (Flash Memory):固态硬盘 or 闪存
 - Magnetic disk (Hard disk) : 硬盘

Memory		Price	Speed	Capacity	Used for
SRAM	volatile	/	Fastest	KB~MB	Cache
DRAM	volatile	\$3~4/GB	Fast	MB~GB	Main memory
Hard disk	nonvolatile	\$0.05~1/GB	Slow	TB~PB	PC Storage
Flash Memory	nonvolatile	\$0.75~\$1/GB	Medium	GB~TB	PMD Storage

Computer organization

□ Decomposability of computer systems



Software categorization

□ Categorize software by its use

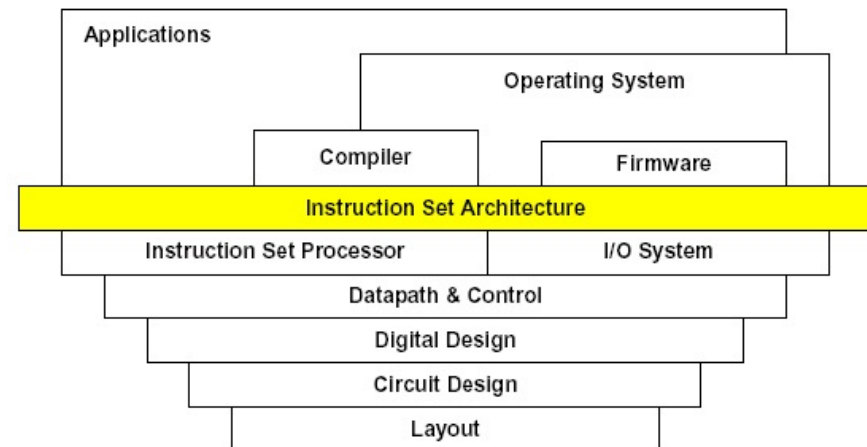
- Systems software ---- aimed at programmers
- Applications software ---- aimed at users

□ Operating system

- Handling basic input and output operations
- Allocating storage and memory
- Sharing the computer among multiple applications using it simultaneously

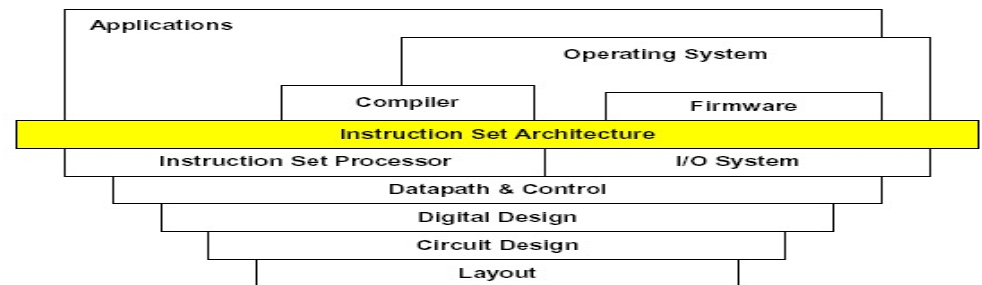
□ **Compiler: Translation of a program written in HLL(高级编程语言)**

□ **Firmware(驱动): Software specially designed for a piece of hardware**



From a High-Level Language to the Language of Hardware

- ❑ Lower-level details are hidden to higher levels
- ❑ **Instruction set architecture (指令集)** ---- the interface between hardware and lowest-level software
- ❑ Many implementations of varying cost and performance can run identical software



From a High-Level Language to the Language of Hardware



□ Machine language(机器语言)

- Computers only understands electrical signalson/off
- Binary numbers express machine instructions
ex. 1000110010100000 means to add two numbers

□ Assembly language(汇编语言)

- Symbolic notations ex. add A, B
- The assembler translates them into machine instruction

□ High-level programming language(高级编程语言)

- Notations more closer to the natural language
ex. A + B
- The compiler translates into assembly language
- Advantages over assembly language
 - Think in a more natural language
 - Programs can be independent of hardware

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```



C compiler



Assembly
language
program
(for MIPS)

```
swap:
  muli $2, $5,4
  add $2, $4,$2
  lw $15, 0($2)
  lw $16, 4($2)
  sw $16, 0($2)
  sw $15, 4($2)
  jr $31
```



Assembler

Binary machine
language
program
(for MIPS)

```
000000001010000100000000000011000
00000000100011100001100000100001
10001100011000100000000000000000
100011001111001000000000000000100
10101100111100100000000000000000
101011000110001000000000000000100
00000011111000000000000000001000
```



Outline

- Introduction
- Computer organization
- **How build processors ?**
- Computer design: performance and idea
- What you can learn from this course ?

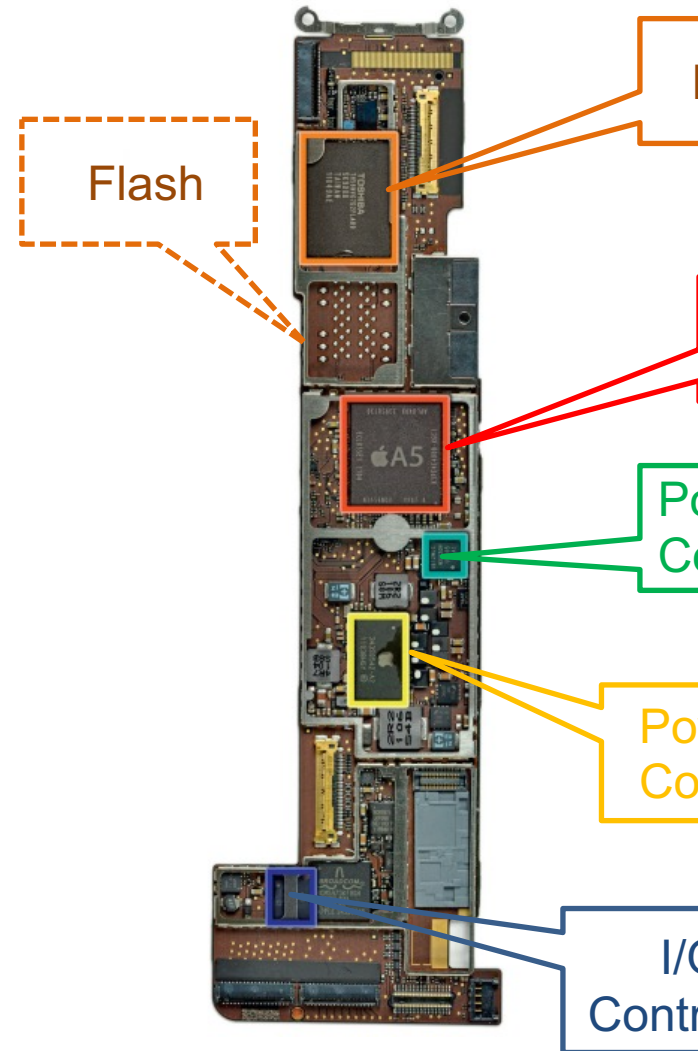
Integrated circuits

- Integrated circuits the kernel components of the logic boards: driving our advancing technology.

(主板核心由集成电路/芯片构成，集成电路驱动我们技术的进步)

- Both processor and memory are made of integrated circuits.

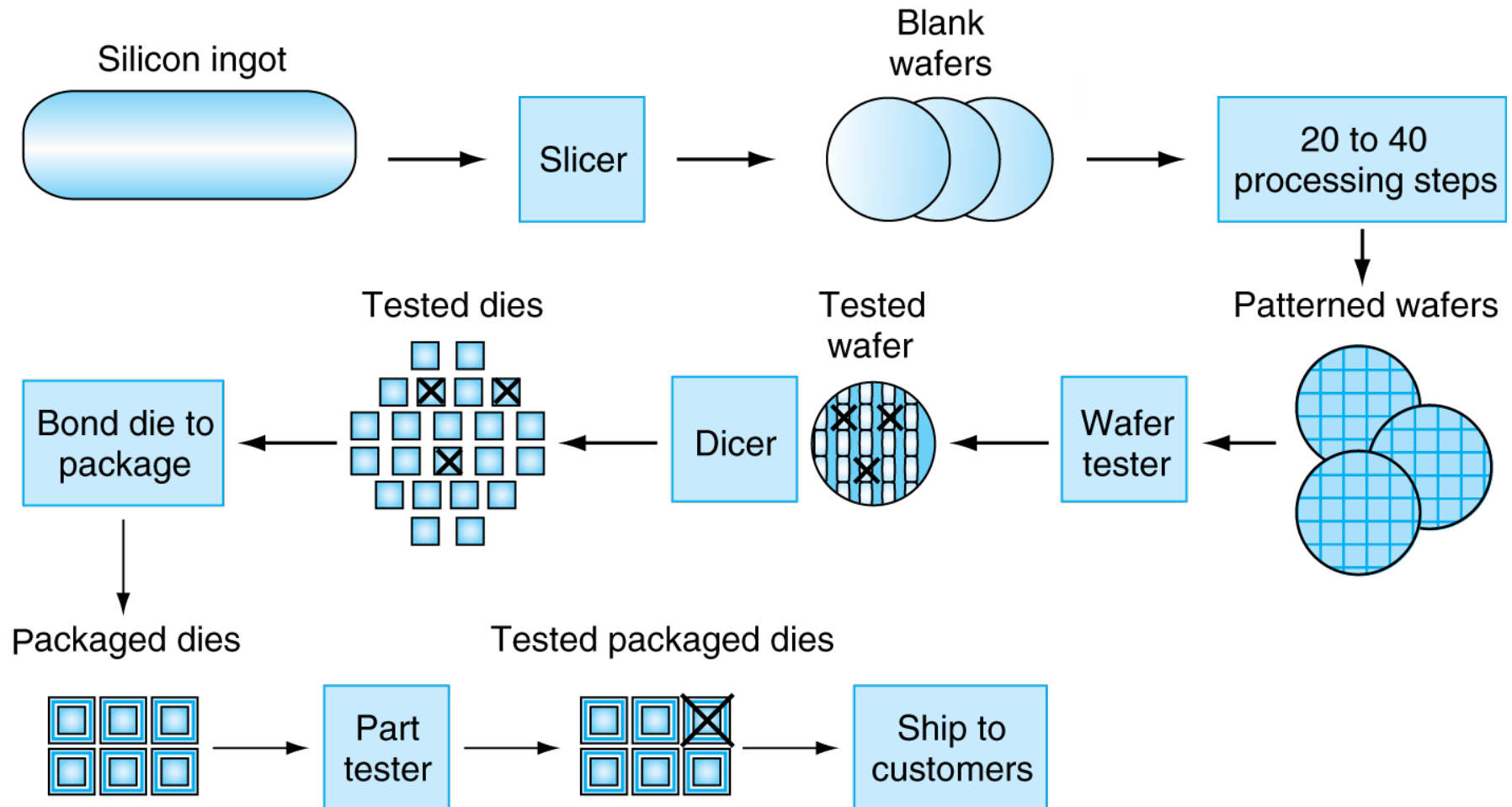
How to build
?



Manufacturing ICs



□ The chip manufacturing process





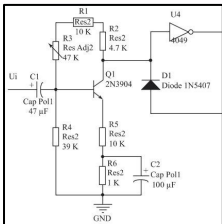
(集成电路制造流程)

芯片设计
Chip design

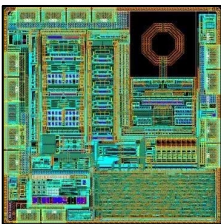
晶圆生产
Wafer production

芯片封装
Chip package

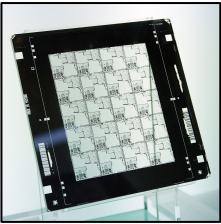
芯片测试
Chip test



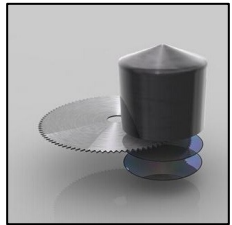
Circuit design(电路设计)



Layout design(版图设计)



Mask(掩膜)



Ingots cutting(铸锭切割)

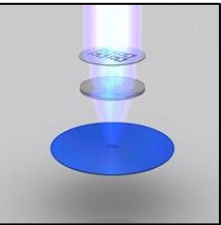
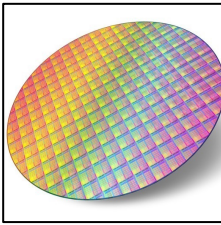
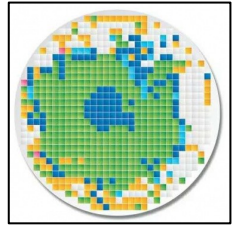


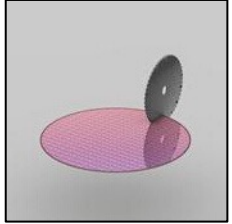
Photo resist(光刻)



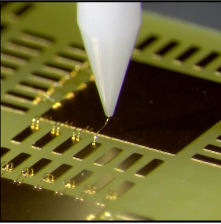
Wafer(晶圆)



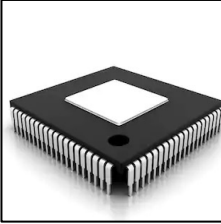
Wafer test(晶圆测试)



Slicing(晶圆切片)



Wire Bond(焊线)



Package(封装)



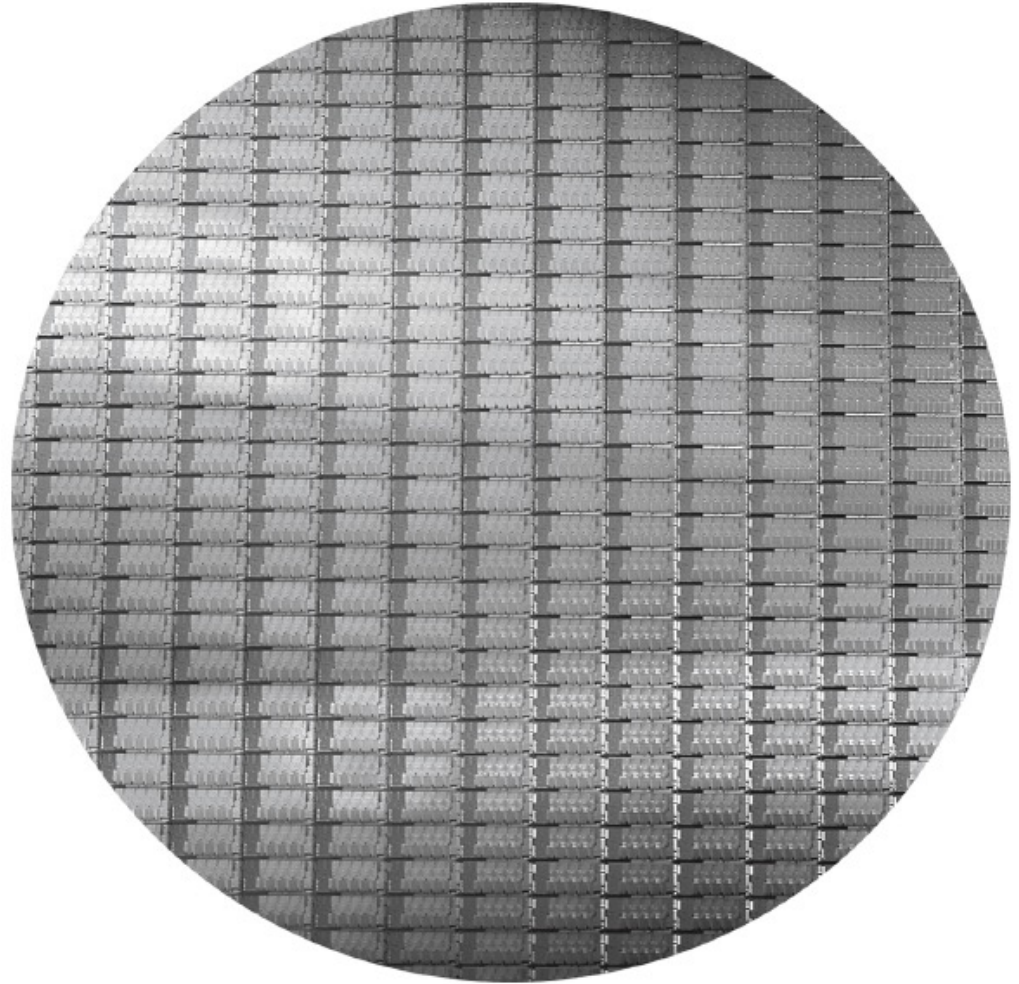
Final test(成品测试)



Finished chip(芯片成品)

Intel Core i7 Wafer

- 300mm wafer,
280 chips,
32nm technology
- Each chip is:
 20.7×10.5 mm





Integrated Circuit Cost

□ Yield: proportion of working dies per wafer

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area} / 2))^2}$$

□ Nonlinear relation to area and defect rate

- Wafer cost and area are fixed
- Defect rate determined by manufacturing process
- Die area determined by architecture and circuit design

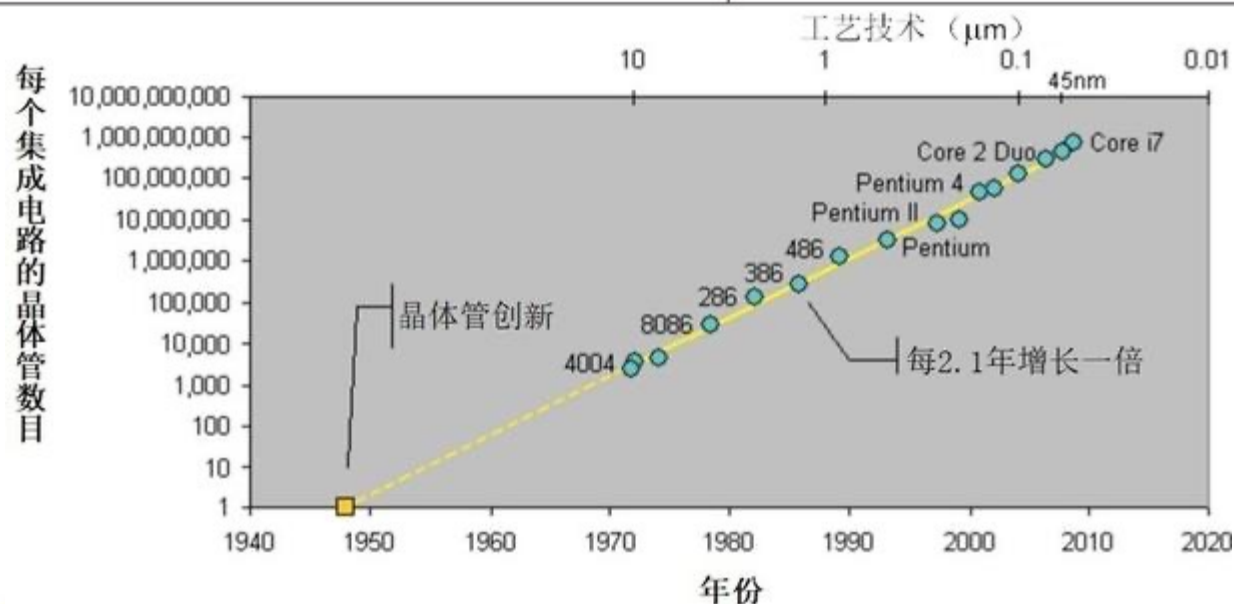
IC procedures



Development of integrated circuits (集成电路发展)

Relative performance per unit cost of technologies used in computers over time

Year	Technology used in computers	Relative performance/unit cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit	900
1995	Very large-scale integrated circuit	2,400,000
2013	Ultra large-scale integrated circuit	250,000,000,000



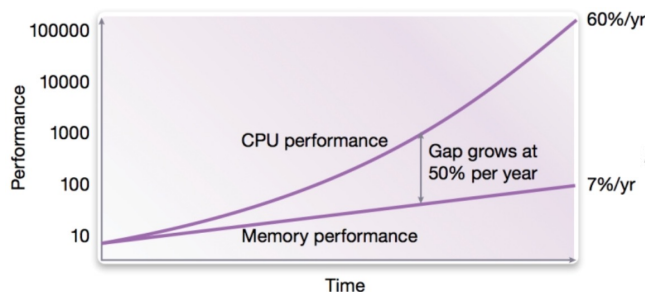
Challenges of integrated circuits manufacturing



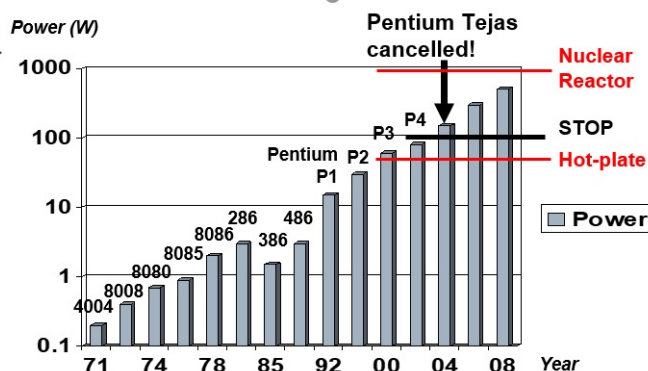
- ❑ The integration of SoC increases rapidly(1billions transistor on single chip), and complexity augments increasingly.
(集成度越来越高，复杂度快速增加)
- ❑ The size of the IC process reaches physical limitation(5nm now). More and more precision requirements and more and more difficult to manufacture.
(工艺尺寸达到物理极限，加工精度越来越高，制造难度越来越大)
- ❑ Involved many technologies: Electronics, optics, Mechanics, computer science... Each technology is hard to break through.
(多个技术的集成：电子，光学，机械，计算机，每一个技术都能难突破)

Challenges of the processor

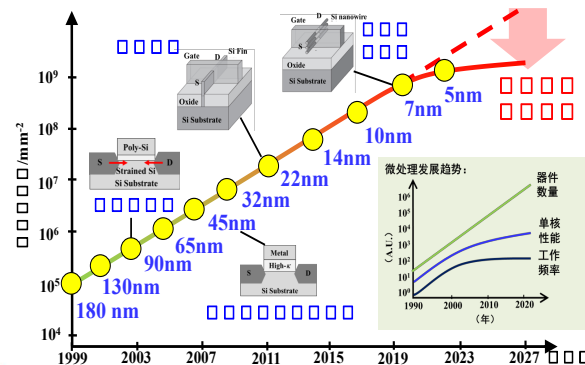
- ❑ 处理器和存储器性能的提高不成比例，“内存墙”问题日趋严重，“冯诺依曼”架构面领挑战。
- ❑ 单位面积功耗增大，散热问题日趋严峻，“功耗墙”问题凸显。
- ❑ 受物理规律限制，摩尔定律面临失效，靠工艺进步获得的性能红利逐渐减弱。



Memory Wall (内存墙)



Power Wall(功耗墙)



工艺发展限制

Outline



- Introduction
- Computer organization
- How build processors ?
- **Computer design: performance and idea**
- What you can learn from this course ?



Response Time and Throughput

□ Response time/execution time

响应时间/执行时间

- How long it takes to do a task

□ Throughput (bandwidth)

吞吐率

- Total work done per unit time
 - e.g., tasks/transactions/... per hour

□ How are response time and throughput affected by

- Replacing the processor with a faster version?
- Adding more processors?

□ We'll focus on response time for now...



Relative Performance

- Define Performance = 1/Execution Time
- “X is n time faster than Y”

$$\begin{aligned} & \text{Performance}_X / \text{Performance}_Y \\ &= \text{Execution time}_Y / \text{Execution time}_X = n \end{aligned}$$

□ Example: time taken to run a program

- 10s on A, 15s on B
- $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15\text{s} / 10\text{s} = 1.5$
- So A is 1.5 times faster than B



Measuring Execution Time

□ Elapsed time

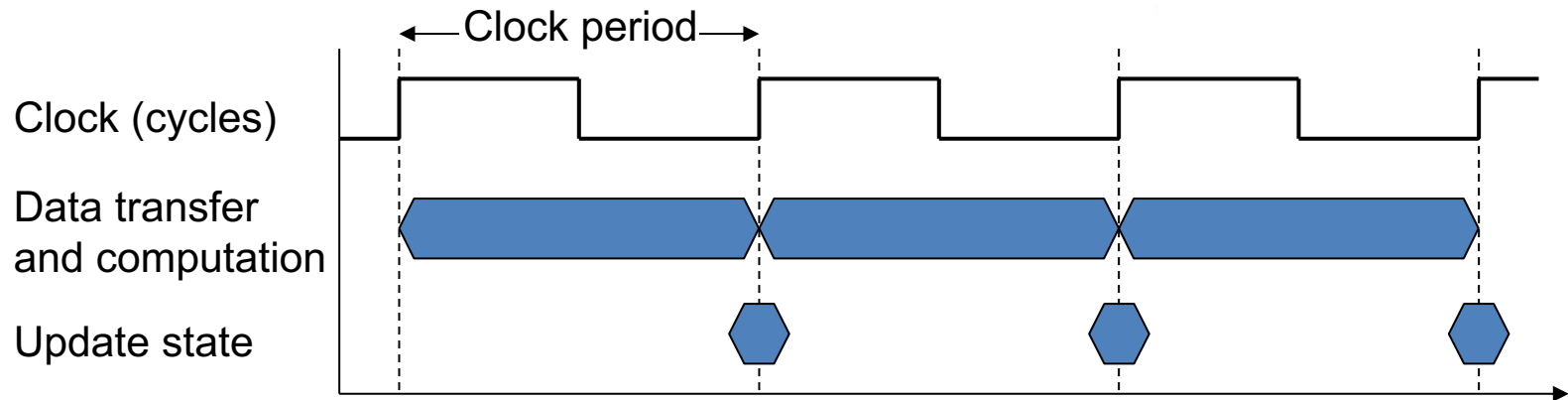
- Total response time, including all aspects
 - Processing, I/O, OS overhead, idle time
- Determines system performance

□ CPU time

- Time spent processing a given job
 - Discounts I/O time, other jobs' shares
- Comprises user CPU time and system CPU time
- Different programs are affected differently by CPU and system performance

Measuring Execution Time

❑ Operation of digital hardware governed by a constant-rate clock



❑ Clock period: duration of a clock cycle

- e.g., $250\text{ps} = 0.25\text{ns} = 250 \times 10^{-12}\text{s}$

❑ Clock frequency (rate): cycles per second

- e.g., $4.0\text{GHz} = 4000\text{MHz} = 4.0 \times 10^9\text{Hz}$



CPU Time

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

□ Performance improved by

- Reducing number of clock cycles
- Increasing clock rate
- Hardware designer must often trade off clock rate against cycle count



CPU Time Example

- ❑ Computer A: 2GHz clock, 10s CPU time
- ❑ Designing Computer B
 - Aim for 6s CPU time
 - Can do faster clock, but causes $1.2 \times$ clock cycles
- ❑ How fast must Computer B clock be?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6s}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10s \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6s} = \frac{24 \times 10^9}{6s} = 4\text{GHz}$$



Instruction Count and CPI

Clock Cycles = Instruction Count $\times$ Cycles per Instruction

CPU Time = Instruction Count $\times$ CPI $\times$ Clock Cycle Time

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

❑ Instruction Count for a program

- Determined by program, ISA and compiler

❑ Average cycles per instruction

- Determined by CPU hardware
- If different instructions have different CPI
 - ❑ Average CPI affected by instruction mix



CPI Example

- ❑ Computer A: Cycle Time = 250ps, CPI = 2.0
- ❑ Computer B: Cycle Time = 500ps, CPI = 1.2
- ❑ Same ISA
- ❑ Which is faster, and by how much?

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps} \end{aligned}$$

A is faster...

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps} \end{aligned}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2$$

...by this much



CPI in More Detail

- ❑ If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

- ❑ Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}} \right)$$

Relative frequency

CPI Example

- Alternative compiled code sequences using instructions in classes A, B, C

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Sequence 1: IC = 5

- Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$
- Avg. CPI = $10/5 = 2.0$

- Sequence 2: IC = 6

- Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$
- Avg. CPI = $9/6 = 1.5$



Performance summary

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

□ Performance depends on

- Algorithm: affects IC, possibly CPI
- Programming language: affects IC, CPI
- Compiler: affects IC, CPI
- Instruction set architecture: affects IC, CPI, T_c

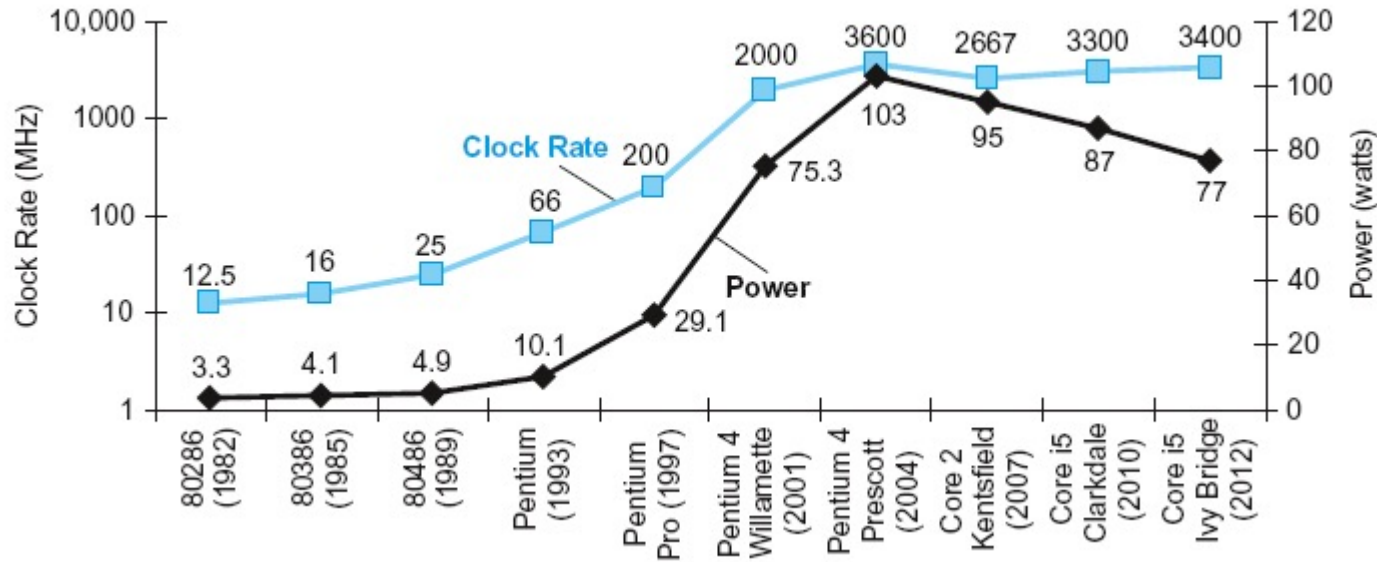


Where is the performance bottleneck?

Hardware or software component	How this component affects performance	Where is this topic covered?
Algorithm	Determines both the number of source-level statements and the number of I/O operations executed	Other books
Programming language, compiler, and architecture	Determines the number of machine instructions for each source-level statements	Other books
Processor and memory system	Determines how fast instructions can be executed	Chapter 4, 5
I/O system(hardware and operating system)	Determines how fast I/O operations may be executed	Other books

What we could do ?

Power Trends



□ In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

× 30

5V → 1V

× 1000



CPI in More Detail

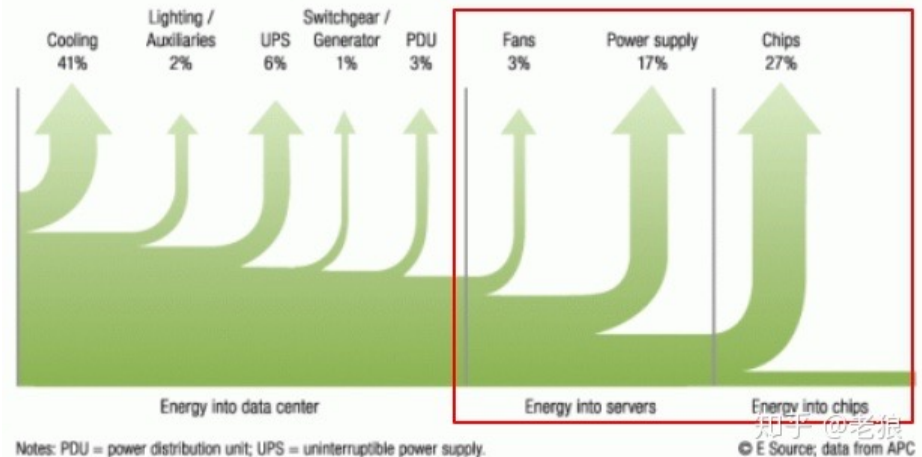
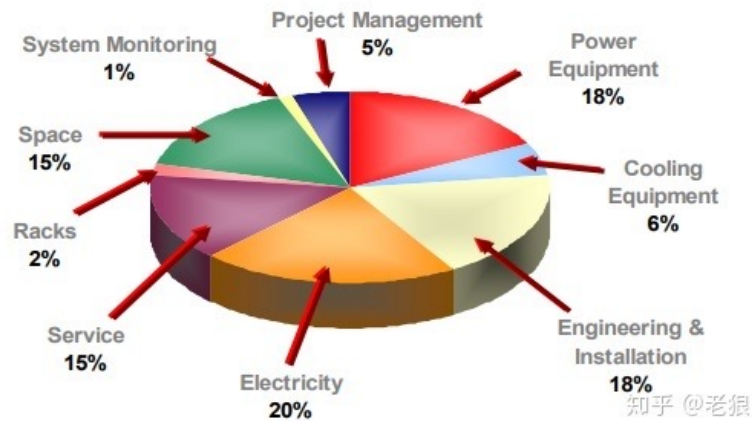
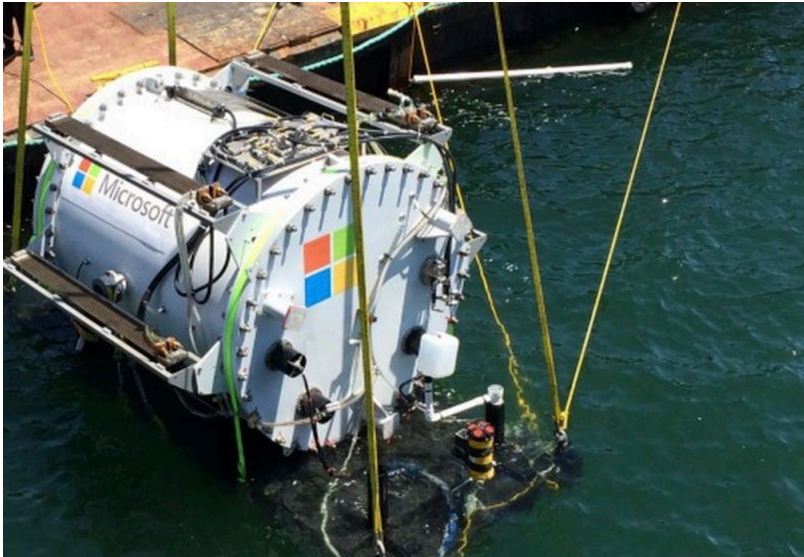
□ Suppose a new CPU has

- 85% of capacitive load of old CPU
- 15% voltage and 15% frequency reduction

$$\frac{P_{\text{new}}}{P_{\text{old}}} = \frac{C_{\text{old}} \times 0.85 \times (V_{\text{old}} \times 0.85)^2 \times F_{\text{old}} \times 0.85}{C_{\text{old}} \times V_{\text{old}}^2 \times F_{\text{old}}} = 0.85^4 = 0.52$$

□ The power wall

- We can't reduce voltage further
- We can't remove more heat







Multiprocessors

❑ Multicore microprocessors

- More than one processor per chip

❑ Requires explicitly parallel programming

- Compare with instruction level parallelism
 - ❑ Hardware executes multiple instructions at once
 - ❑ Hidden from the programmer
- Hard to do
 - ❑ Programming for performance
 - ❑ Load balancing
 - ❑ Optimizing communication and synchronization



SPEC CPU Benchmark

- ❑ **Programs used to measure performance**
 - Supposedly typical of actual workload
- ❑ **Standard Performance Evaluation Corp (SPEC)**
 - Develops benchmarks for CPU, I/O, Web, ...
- ❑ **SPEC CPU2006**
 - Elapsed time to execute a selection of programs
 - ❑ Negligible I/O, so focuses on CPU performance
 - Normalize relative to reference machine
 - Summarize as geometric mean of performance ratios
 - ❑ CINT2006 (integer) and CFP2006 (floating-point)

$$\sqrt[n]{\prod_{i=1}^n \text{Execution time ratio}_i}$$



CINT2006 for Intel Core i7 920

Description	Name	Instruction Count x 10 ⁹	CPI	Clock cycle time (seconds x 10 ⁻⁹)	Execution Time (seconds)	Reference Time (seconds)	SPECratio
Interpreted string processing	perl	2252	0.60	0.376	508	9770	19.2
Block-sorting compression	bzip2	2390	0.70	0.376	629	9650	15.4
GNU C compiler	gcc	794	1.20	0.376	358	8050	22.5
Combinatorial optimization	mcf	221	2.66	0.376	221	9120	41.2
Go game (AI)	go	1274	1.10	0.376	527	10490	19.9
Search gene sequence	hmmer	2616	0.60	0.376	590	9330	15.8
Chess game (AI)	sjeng	1948	0.80	0.376	586	12100	20.7
Quantum computer simulation	libquantum	659	0.44	0.376	109	20720	190.0
Video compression	h264avc	3793	0.50	0.376	713	22130	31.0
Discrete event simulation library	omnetpp	367	2.10	0.376	290	6250	21.5
Games/path finding	astar	1250	1.00	0.376	470	7020	14.9
XML parsing	xalancbmk	1045	0.70	0.376	275	6900	25.1
Geometric mean	-	-	-	-	-	-	25.7



SPEC Power Benchmark

□ Power consumption of server at different workload levels

- Performance: ssj_ops/sec (server side java business application)
- Power: Watts (Joules/sec)

$$\text{Overall ssj_ops per Watt} = \left(\sum_{i=0}^{10} \text{ssj_ops}_i \right) / \left(\sum_{i=0}^{10} \text{power}_i \right)$$



SPECpower_ssj2008 for Xeon X5650

Target Load %	Performance (ssj_ops)	Average Power (Watts)
100%	865,618	258
90%	786,688	242
80%	698,051	224
70%	607,826	204
60%	521,391	185
50%	436,757	170
40%	345,919	157
30%	262,071	146
20%	176,061	135
10%	86,784	121
0%	0	80
Overall Sum	4,787,166	1,922
$\Sigma ssj_ops / \Sigma power =$		2,490



Pitfall: Amdahl's Law

- ❑ Improving an aspect of a computer and expecting a proportional improvement in overall performance

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- ❑ Example: multiply accounts for 80s/100s
 - How much improvement in multiply performance to get 5× overall?

$$20 = \frac{80}{n} + 20$$

■ Can't be done!

- ❑ Corollary: make the common case fast



Fallacy: Low Power at Idle

❑ Look back at i7 power benchmark

- At 100% load: 258W
- At 50% load: 170W (66%)
- At 10% load: 121W (47%)

❑ Google data center

- Mostly operates at 10% – 50% load
- At 100% load less than 1% of the time

❑ Consider designing processors to make power proportional to load



Pitfall: MIPS as a Performance Metric

❑ MIPS: Millions of Instructions Per Second

- Doesn't account for
 - ❑ Differences in ISAs between computers
 - ❑ Differences in complexity between instructions

$$\begin{aligned}\text{MIPS} &= \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} \\ &= \frac{\text{Instruction count}}{\frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}} \times 10^6 = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}\end{aligned}$$

- CPI varies between programs on a given CPU



Concluding Remarks

- ❑ **Cost/performance is improving**
 - Due to underlying technology development
- ❑ **Hierarchical layers of abstraction**
 - In both hardware and software
- ❑ **Instruction set architecture**
 - The hardware/software interface
- ❑ **Execution time: the best performance measure**
- ❑ **Power is a limiting factor**
 - Use parallelism to improve performance



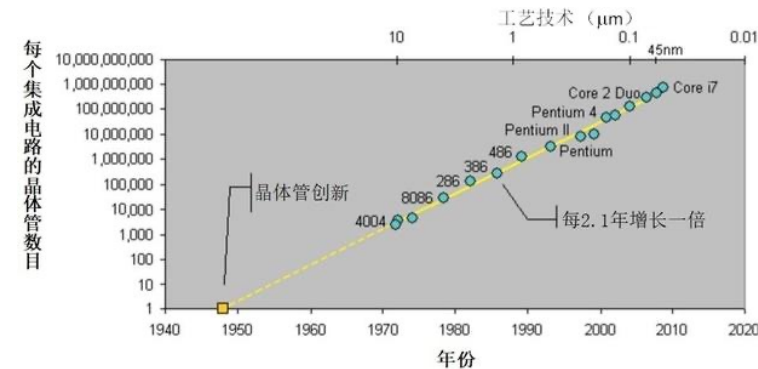
Eight Great Ideas

- ❑ Design for Moore's Law (设计紧跟摩尔定律)
- ❑ Use Abstraction to Simplify Design (采用抽象简化设计)
- ❑ Make the Common Case Fast (加速大概率事件)
- ❑ Performance via Parallelism (通过并行提高性能)
- ❑ Performance via Pipelining (通过流水线提高性能)
- ❑ Performance via Prediction (通过预测提高性能)
- ❑ Hierarchy of Memories (存储器层次)
- ❑ Dependability via Redundancy (通过冗余提高可靠性)

Idea1: Design for Moore's law

- ❑ One **constant** for computer designer is **rapid change**.
- ❑ Design for **where it will be** when finishes rather than design for where it starts.

Gordon Moore, a cofounder of Intel, predicts that the number of transistors which can be placed on a single chip will double every 2 year. It has held steady and was dubbed “**Moore's Law**” around 1970

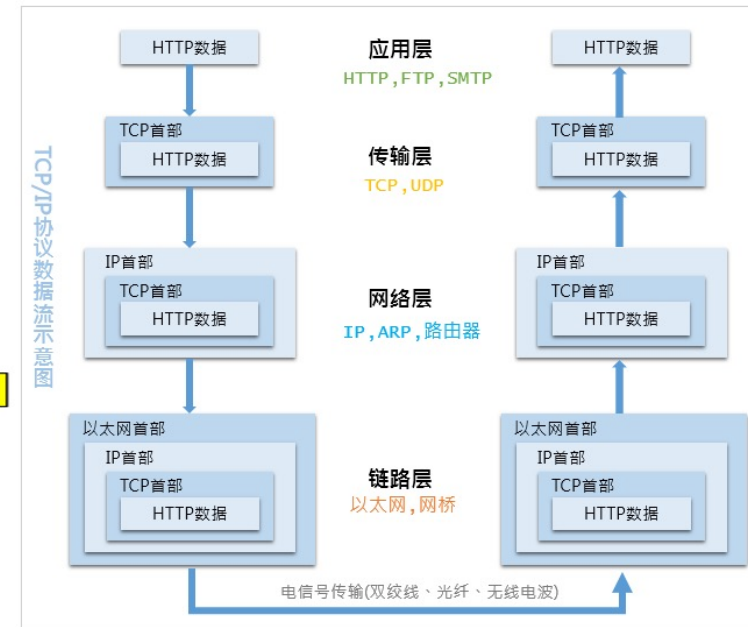
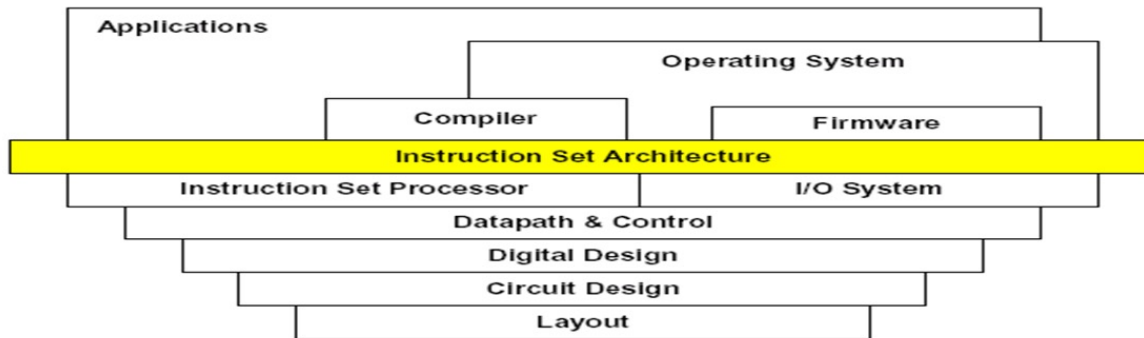


Why Moore's law?

- ❑ Smaller Transistor, **more function** on same chip area;
- ❑ Smaller Transistor, **low power**
- ❑ Smaller Transistor, smaller area for same function, **low cost**;

Idea2: Use abstraction to simplify design

- Lower-level details are hidden to offer a simple model at higher level





Idea3: Make the common case fast

- ❑ Making the common case fast will tend to enhance performance better than optimizing the rare case

```
for(i=0;i<2;i++){  
    洗衣半小时  
}
```

```
for(i=0;i<1000;i++){  
    上班走路1小时  
}
```

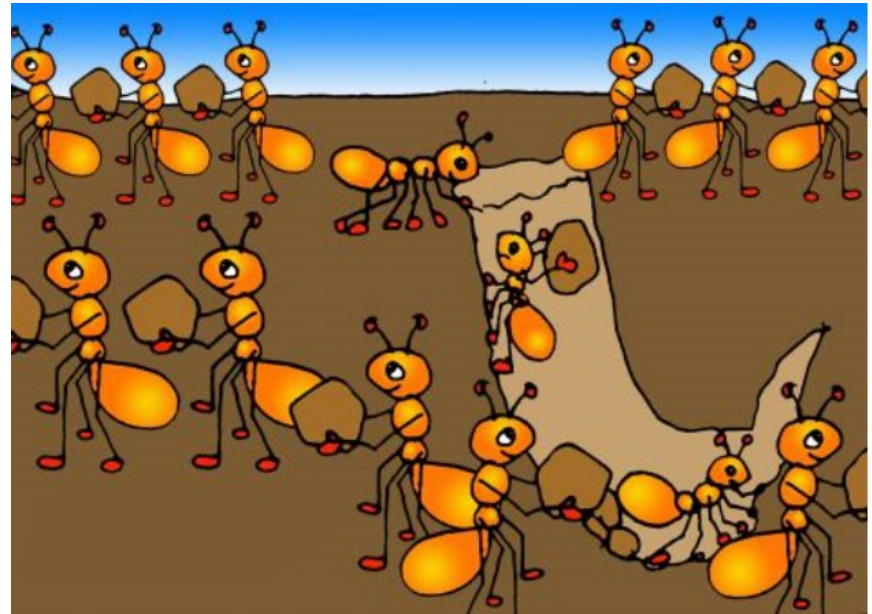
I think I should buy a car first.

```
for(i=0;i<2;i++){  
    做饭半小时  
}
```

Idea4: Performance via parallelism



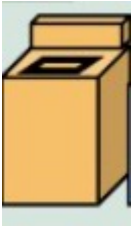
- Get more performance by performing operation in parallel



Idea5: Performance via pipelining

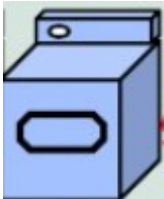


- A, B, C, D all have some clothes to wash, dryer and fold



Wash: 30minuts

Total time ?



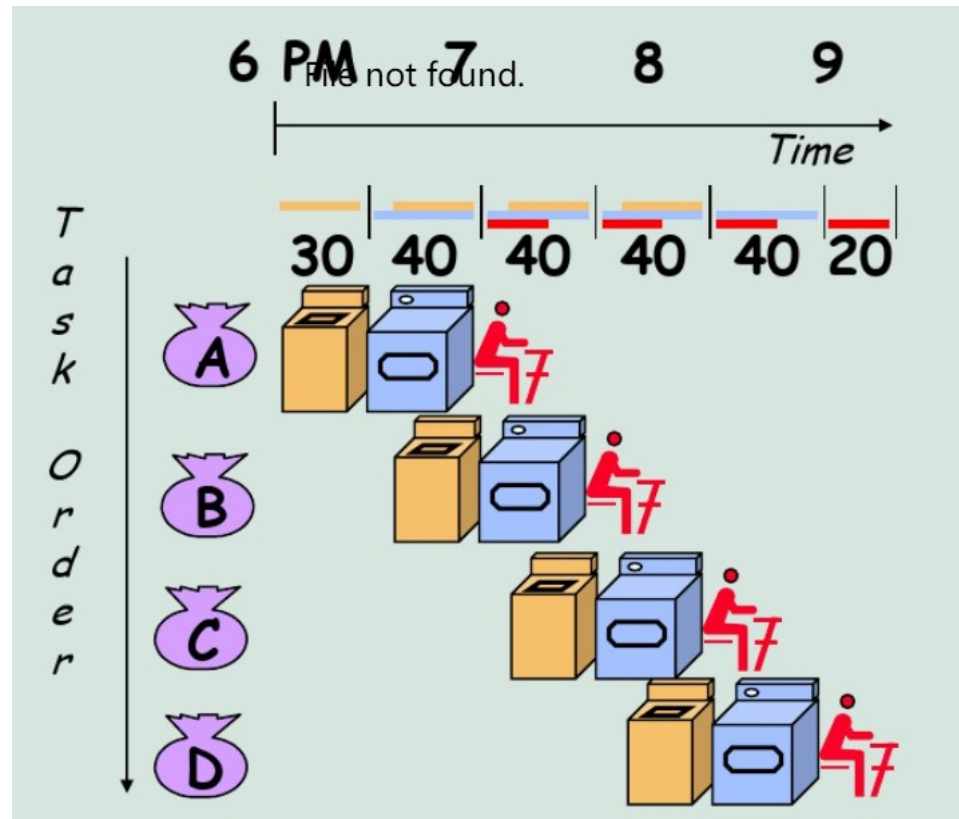
Dryer: 40minuts

$$(30+40+20) \times 4 = 360$$



Fold: 20minuts

Idea5 :Performance via pipelining



$$30+40*4+20=210$$

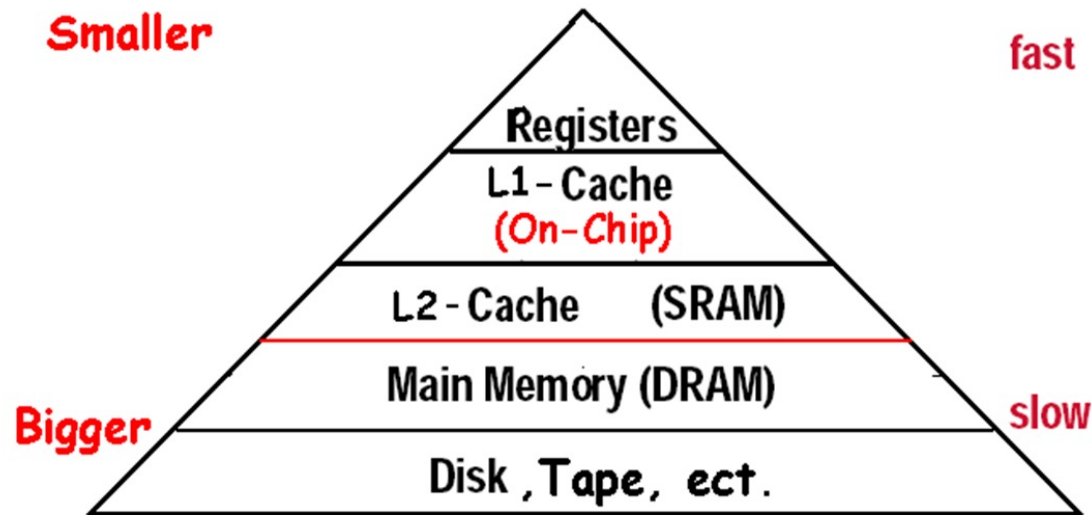
Idea6 :Performance via prediction

- ❑ To guess and start working rather than waiting until you know for sure
- ❑ The mechanism to recover from a misprediction is not too expensive and prediction is relatively accurate



Idea7 :Hierarchy of memories

- With the fastest, smallest, and most expensive memory per bit at the top of the hierarchy and the slowest, largest, and cheapest per bit at the bottom



Idea8: Dependability via redundancy

- ❑ Not only be fast, also to be **dependable**.
- ❑ Any physical device can fail.
- ❑ Make systems **dependable** by including redundant components that can take over when a failure occurs *and* to help detect failures





Outline

- Introduction
- Computer organization
- How build processors ?
- Computer design: performance and idea
- What you can learn from this course ?

Content



- ❑ Chapter1: Computer Abstraction and Technology
- ❑ Chapter2: Instructions: Language of the Computer
- ❑ Chapter3: Arithmetic for Computers
- ❑ Chapter4: The Processor: Datapath and Control
- ❑ Chapter5: Large and Fast: Exploiting Memory Hierarchy
- ❑ Chapter 6: Parallel processor from client to Cloud (选讲, 非考试内容)
- ❑ Appendix: Storage, Networks, and Other Peripherals (Ch8 of Version 3, 了解概念)

核心内容



After the course, you should know

- ❑ **The internal organization of computers and its influence on the performance of programs (处理器内部组织结构及其性能影响)**
- ❑ **The hierarchy of software and hardware**
 - How are programs written in high-level language translated into the language of the hardware, and how does it run?
(高级语言编写的程序如何变成硬件的语言, 它是如何工作的?)
 - What is the interface between the software and the hardware, and how does software instruct the hardware to perform?
(软硬件之间的接口是什么, 软件如何指导硬件工作)
 - What determines the performance of a program, and a programmer improve the performance?(什么决定了一个程序的性能)
 - What techniques can used to improve performance?
(什么技术我们可以用来提高性能)



Homework

□ 1.2, 1.5, 1.6, 1.7, 1.13



◎ END